AI Agent 전문 개발 과정

Day 1 — 문제 정의 & 설계 전략

MCP · RAG · Hybrid 아키텍처 기반의 실무형 AI Agent 설계

5일 과정 (40시간) · 강의 30% / 실습 70%

5일 커리큘럼

Day 1	Day 2	Day 3	Day 4	Day 5
문제 정의 & 설계	제어 흐름 & 상태	MCP · RAG 구현	평가 · 운영 전략	MVP Project
Pain→Task→Tool 프롬프트 전략 MCP·RAG 판단	LangGraph Tool Validation 리팩토링	MCP 고급 설계 RAG 튜닝 Hybrid	품질 평가 모니터링 서비스 아키텍처	MVP 설계 구현·시연 발표

Day 1 시간표

교시	시간	주제	실습
1	09:00–11:00	Agent 문제 정의와 과제 도출	Agent 후보 도출 워크시트
2	11:00–13:00	LLM 동작 원리 · 프롬프트 전략	컨텍스트 전략 비교
	13:00–14:00	점심시간	
3	14:00–16:00	Agent 기획서 구조화	Agent 구조 다이어그램
4	16:00–18:00	MCP · RAG · Hybrid 구조 판단	MCP vs RAG 비교 구현

환경 세팅

```
# 1. uv 설치 (Linux / WSL)
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env

# 2. 의존성 설치 (day1/ 디렉토리에서)
cd labs/day1
uv sync

# 3. 환경변수 설정
cp .env.example .env
# .env에 OPENAI_API_KEY=sk-... 입력

# 4. Jupyter Lab 실행
uv run jupyter lab
```

uv: Rust 기반 Python 패키지 매니저 — pip보다 10~100배 빠름

1교시

Agent 문제 정의와 과제 도출

Pain → Task → Skill → Tool

Agent가 모든 문제의 해답이 아니다

Agent가 모든 문제의 해답이 아니다

LLM 기술이 발전하면서 "**Agent로 다 해결**"이라는 기대가 높아졌다

Agent가 모든 문제의 해답이 아니다

LLM 기술이 발전하면서 "**Agent로 다 해결**"이라는 기대가 높아졌다

하지만 잘못된 문제에 Agent를 적용하면 **복잡도만 증가**

Agent가 모든 문제의 해답이 아니다

LLM 기술이 발전하면서 "**Agent로 다 해결**"이라는 기대가 높아졌다

하지만 잘못된 문제에 Agent를 적용하면 **복잡도만 증가**

문제 정의가 곧 프로젝트 성패를 결정한다

Agent가 모든 문제의 해답이 아니다

LLM 기술이 발전하면서 "**Agent로 다 해결**"이라는 기대가 높아졌다

하지만 잘못된 문제에 Agent를 적용하면 **복잡도만 증가**

문제 정의가 곧 프로젝트 성패를 결정한다

Agent 프로젝트 실패의 가장 큰 이유:

"왜 Agent가 필요한지" 명확히 정의하지 않고 시작

Agent가 빛을 발하는 조건

Agent가 빛을 발하는 조건

- **다단계 추론 필요** — 여러 단계를 순차적으로 처리 (수집 → 분석 → 보고서)

Agent가 빛을 발하는 조건

- **다단계 추론 필요** — 여러 단계를 순차적으로 처리 (수집 → 분석 → 보고서)
- **도구 사용 필요** — 외부 API, DB, 파일 시스템 접근 (검색 + 계산 + 저장)

Agent가 빛을 발하는 조건

- **다단계 추론 필요** — 여러 단계를 순차적으로 처리 (수집 → 분석 → 보고서)
- **도구 사용 필요** — 외부 API, DB, 파일 시스템 접근 (검색 + 계산 + 저장)
- **비결정적 흐름** — 상황에 따라 다음 단계가 달라짐 (에러 처리, 분기)

Agent가 빛을 발하는 조건

- **다단계 추론 필요** — 여러 단계를 순차적으로 처리 (수집 → 분석 → 보고서)
- **도구 사용 필요** — 외부 API, DB, 파일 시스템 접근 (검색 + 계산 + 저장)
- **비결정적 흐름** — 상황에 따라 다음 단계가 달라짐 (에러 처리, 분기)
- **반복 작업 자동화** — 사람이 하면 지루하고 오래 걸리는 작업

Agent가 빛을 발하는 조건

- **다단계 추론 필요** — 여러 단계를 순차적으로 처리 (수집 → 분석 → 보고서)
- **도구 사용 필요** — 외부 API, DB, 파일 시스템 접근 (검색 + 계산 + 저장)
- **비결정적 흐름** — 상황에 따라 다음 단계가 달라짐 (에러 처리, 분기)
- **반복 작업 자동화** — 사람이 하면 지루하고 오래 걸리는 작업

핵심: **외부 시스템 상호작용 + 비결정적 흐름** → Agent 적합

Agent가 과잉인 경우

상황	더 나은 대안
단순 Q&A	일반 챗봇 or **RAG**
고정 템플릿 생성	**프롬프트 엔지니어링**
단일 API 호출	**함수 직접 호출**
정형 데이터 처리	**전통적 파이프라인**

Agent가 과잉인 경우

상황	더 나은 대안
단순 Q&A	일반 챗봇 or **RAG**
고정 템플릿 생성	**프롬프트 엔지니어링**
단일 API 호출	**함수 직접 호출**
정형 데이터 처리	**전통적 파이프라인**

LLM으로 할 수 있다고 해서 **해야 하는 건 아니다**

Pain → Task → Skill → Tool

1단계
Pain
고통

→

2단계
Task
과제

→

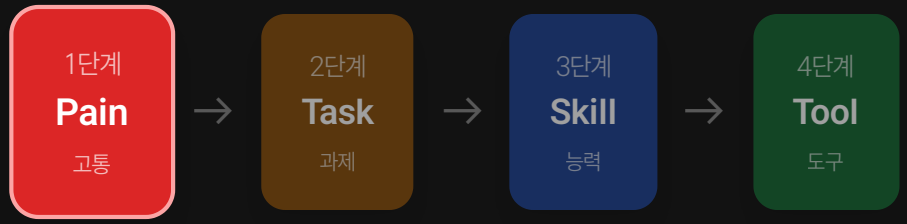
3단계
Skill
능력

→

4단계
Tool
도구

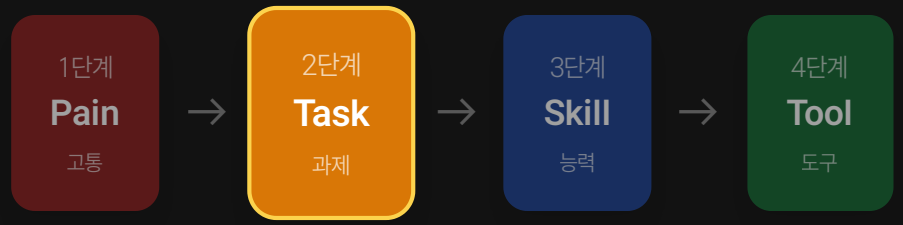
클릭하여 각 단계 확인

Pain → Task → Skill → Tool



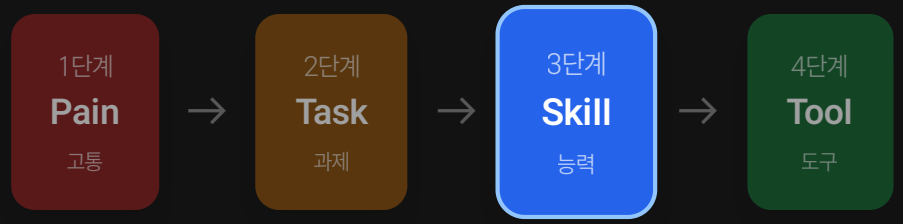
Pain: 업무의 고통을 구체적 수치로 정의 — "매주 3시간 보고서 작성"

Pain → Task → Skill → Tool



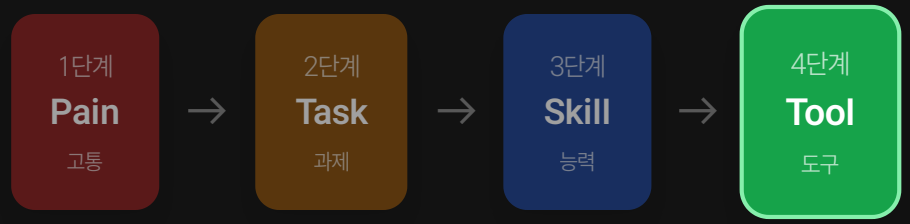
Task: Pain을 해결할 과제 정의 — "데이터 수집 후 요약본 자동 생성"

Pain → Task → Skill → Tool



Skill: 과제 수행에 필요한 능력 — "데이터 수집, 수치 해석, 문서 작성"

Pain → Task → Skill → Tool



Tool: 능력을 구현하는 도구 — "Jira API, DB 쿼리, LLM 요약, Slack 발송"

프레임워크 적용 예시

주간 보고서 자동화

프레임워크 적용 예시

주간 보고서 자동화

Pain 매주 **3시간**을 보고서 수동 작성에 낭비한다

프레임워크 적용 예시

주간 보고서 자동화

Pain 매주 **3시간**을 보고서 수동 작성에 낭비한다

Task 여러 시스템 데이터를 모아 **요약본 자동 생성**

프레임워크 적용 예시

주간 보고서 자동화

Pain 매주 **3시간**을 보고서 수동 작성에 낭비한다

Task 여러 시스템 데이터를 모아 **요약본 자동 생성**

Skill 데이터 수집 · 수치 해석 · 트렌드 파악 · 자연어 요약

프레임워크 적용 예시

주간 보고서 자동화

Pain 매주 **3시간**을 보고서 수동 작성에 낭비한다

Task 여러 시스템 데이터를 모아 **요약본 자동 생성**

Skill 데이터 수집 · 수치 해석 · 트렌드 파악 · 자연어 요약

Tool Jira API · DB Query · LLM 요약 · Slack 발송

프레임워크 적용 예시

주간 보고서 자동화

Pain 매주 **3시간**을 보고서 수동 작성에 낭비한다

Task 여러 시스템 데이터를 모아 **요약본 자동 생성**

Skill 데이터 수집 · 수치 해석 · 트렌드 파악 · 자연어 요약

Tool Jira API · DB Query · LLM 요약 · Slack 발송

성공 기준: 보고서 작성 시간 3h → 5분 이내 단축

업무 유형별 Agent 패턴

업무 유형별 Agent 패턴

자동화형

정해진 프로세스를
자동 실행

트리거 → 처리 → 저장

업무 유형별 Agent 패턴

자동화형

정해진 프로세스를
자동 실행

트리거 → 처리 → 저장

분석형

데이터 탐색하고
인사이트 도출

질문 → 탐색 → 분석 → 설명

업무 유형별 Agent 패턴

⚙️

자동화형

정해진 프로세스를
자동 실행

트리거 → 처리 → 저장

🔍

분석형

데이터 탐색하고
인사이트 도출

질문 → 탐색 → 분석 → 설명

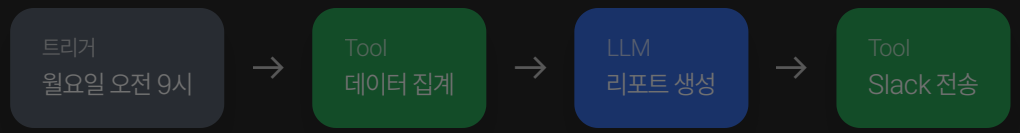
🧠

Planner형

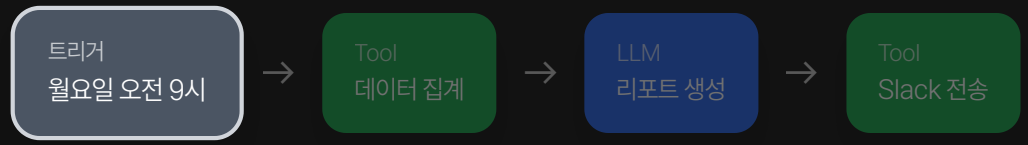
스스로 계획 세우고
실행

목표 → 계획 → 실행 → 통합

자동화형 Agent

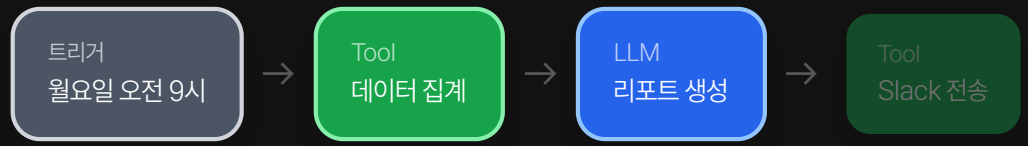


자동화형 Agent



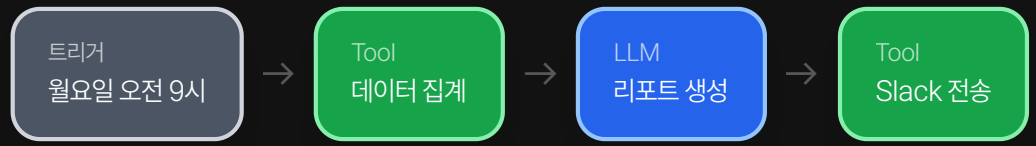
- 반복 주기가 **명확**하다

자동화형 Agent



- 반복 주기가 **명확**하다
- 처리 단계가 **고정**되어 있다

자동화형 Agent



- 반복 주기가 **명확**하다
- 처리 단계가 **고정**되어 있다
- 정상 흐름이 **99%**이다

분석형 Agent

분석형 Agent

- 분석 범위가 사전에 **고정되지 않는다**

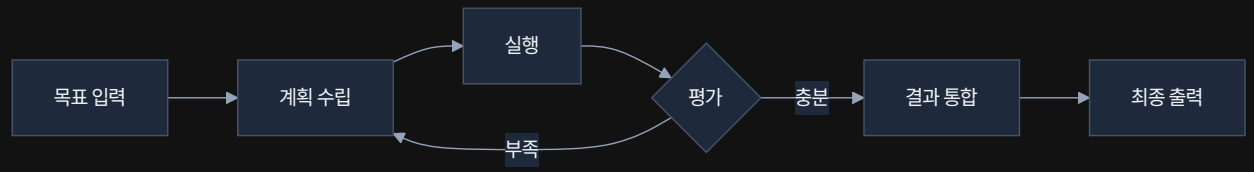
분석형 Agent

- 분석 범위가 사전에 **고정되지 않는다**
- 데이터 소스가 **다양**하다

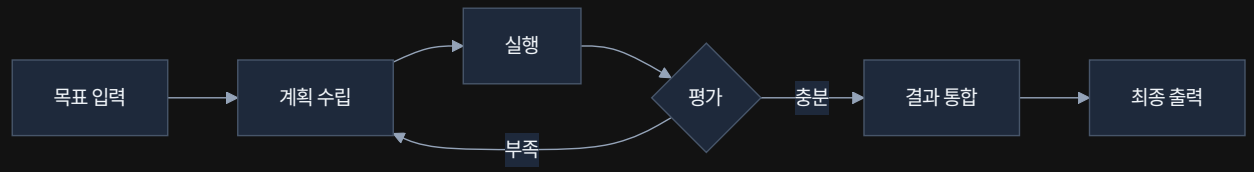
분석형 Agent

- 분석 범위가 사전에 **고정되지 않는다**
- 데이터 소스가 **다양**하다
- 결과를 **자연어**로 설명해야 한다

Planner형 Agent

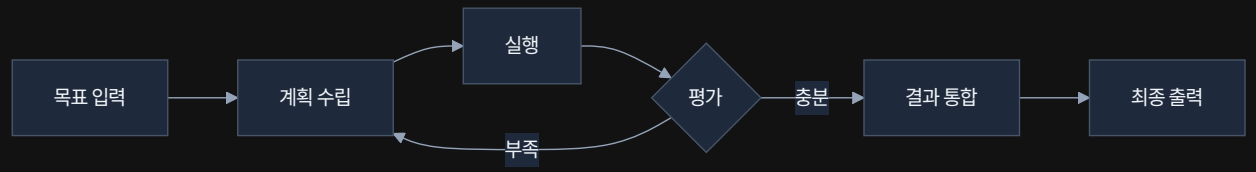


Planner형 Agent



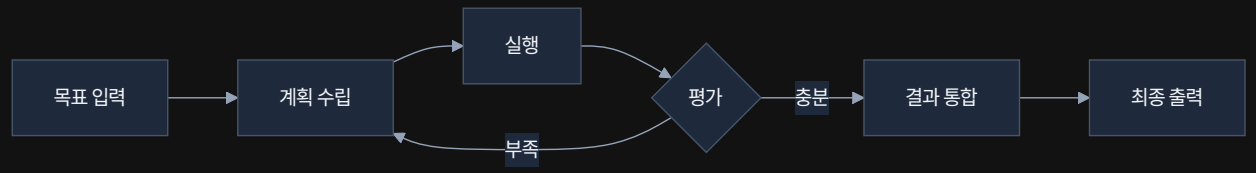
- 목표가 **추상적**이고 실행 방법이 다양하다

Planner형 Agent



- 목표가 **추상적**이고 실행 방법이 다양하다
- 실행 중 **피드백을 반영**해야 한다

Planner형 Agent



- 목표가 **추상적**이고 실행 방법이 다양하다
- 실행 중 **피드백을 반영**해야 한다
- 실패 시 **대안 경로**가 필요하다

3가지 패턴 종합 비교

항목	자동화형	분석형	Planner형
입력	이벤트/스케줄	자연어 질문	목표 설명
계획	고정	반고정	**동적**
도구 수	적음 (1-3)	중간 (3-5)	많음 (5+)
실패 처리	단순 재시도	대안 쿼리	**계획 재수립**
개발 난이도	낮음	중간	**높음**

3가지 패턴 종합 비교

항목	자동화형	분석형	Planner형
입력	이벤트/스케줄	자연어 질문	목표 설명
계획	고정	반고정	**동적**
도구 수	적음 (1-3)	중간 (3-5)	많음 (5+)
실패 처리	단순 재시도	대안 쿼리	**계획 재수립**
개발 난이도	낮음	중간	**높음**

대부분의 실무 Agent는 **자동화형**으로 시작하여 점진적으로 확장한다

RAG vs Agent 판단 기준

RAG

Agent

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성

Agent

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정

Agent

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능

Agent

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순

Agent

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순
- 예: 사내 문서 Q&A

Agent

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순
- 예: 사내 문서 Q&A

Agent

- **핵심 목적**: 작업 수행 + 자동화

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순
- 예: 사내 문서 Q&A

Agent

- **핵심 목적**: 작업 수행 + 자동화
- 다양한 도구 동적 선택

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순
- 예: 사내 문서 Q&A

Agent

- **핵심 목적**: 작업 수행 + 자동화
- 다양한 도구 동적 선택
- Stateful 필요한 경우 많음

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순
- 예: 사내 문서 Q&A

Agent

- **핵심 목적**: 작업 수행 + 자동화
- 다양한 도구 동적 선택
- Stateful 필요한 경우 많음
- 복잡한 흐름 처리

RAG vs Agent 판단 기준

RAG

- **핵심 목적**: 지식 검색 + 생성
- 검색 도구 고정
- Stateless 가능
- 상대적으로 단순
- 예: 사내 문서 Q&A

Agent

- **핵심 목적**: 작업 수행 + 자동화
- 다양한 도구 동적 선택
- Stateful 필요한 경우 많음
- 복잡한 흐름 처리
- 예: 이슈 자동 처리

RAG vs Agent 판단 질문

RAG vs Agent 판단 질문

Q1. 단순히 "찾아서 대답"하면 되는가? → **RAG**

RAG vs Agent 판단 질문

Q1. 단순히 "찾아서 대답"하면 되는가? → RAG

Q2. 외부 시스템과 상호작용이 필요한가? → Agent

RAG vs Agent 판단 질문

Q1. 단순히 "찾아서 대답"하면 되는가? → RAG

Q2. 외부 시스템과 상호작용이 필요한가? → Agent

Q3. 결과를 보고 다음 행동이 달라지는가? → Agent

RAG vs Agent 판단 질문

Q1. 단순히 "찾아서 대답"하면 되는가? → RAG

Q2. 외부 시스템과 상호작용이 필요한가? → Agent

Q3. 결과를 보고 다음 행동이 달라지는가? → Agent

Q4. 문서 기반 지식만 필요한가? → RAG

RAG vs Agent 판단 질문

Q1. 단순히 "찾아서 대답"하면 되는가? → RAG

Q2. 외부 시스템과 상호작용이 필요한가? → Agent

Q3. 결과를 보고 다음 행동이 달라지는가? → Agent

Q4. 문서 기반 지식만 필요한가? → RAG

RAG와 Agent는 **배타적이지 않다** — Agent가 RAG를 도구로 사용하는 Hybrid 구조가 많다

Agent Anti-Pattern

Agent Anti-Pattern

① **망치를 들면 모든 것이 못으로 보인다**

단순 규칙 기반이 더 빠르고 정확한 경우가 많다

Agent Anti-Pattern

① **망치를 들면 모든 것이 못으로 보인다**

단순 규칙 기반이 더 빠르고 정확한 경우가 많다

② **기술로 시작하는 설계**

"API를 써서 뭔가 만들자" → 실패 확률 높음

Agent Anti-Pattern

① **망치를 들면 모든 것이 못으로 보인다**

단순 규칙 기반이 더 빠르고 정확한 경우가 많다

② **기술로 시작하는 설계**

"API를 써서 뭔가 만들자" → 실패 확률 높음

③ **Edge Case 무시**

설계 단계에서 실패 케이스를 미리 나열하라

Agent Anti-Pattern

① **망치를 들면 모든 것이 못으로 보인다**

단순 규칙 기반이 더 빠르고 정확한 경우가 많다

② **기술로 시작하는 설계**

"API를 써서 뭔가 만들자" → 실패 확률 높음

③ **Edge Case 무시**

설계 단계에서 실패 케이스를 미리 나열하라

④ **평가 기준 없는 배포**

정량 지표를 먼저 정의: 정확도, 완료율, 처리 시간

❓ Q&A — Session 1

Q. Agent와 RPA의 차이는?

❓ Q&A — Session 1

Q. Agent와 RPA의 차이는?

> A. RPA는 정해진 UI 흐름을 반복 실행한다. Agent는 상황을 이해하고 유연하게 판단한다.
> RPA는 UI가 바뀌면 깨지지만, Agent는 목표 기반으로 대응한다.

❓ Q&A — Session 1

Q. Agent와 RPA의 차이는?

> A. RPA는 정해진 UI 흐름을 반복 실행한다. Agent는 상황을 이해하고 유연하게 판단한다.
> RPA는 UI가 바뀌면 깨지지만, Agent는 목표 기반으로 대응한다.

❓ Q&A — Session 1

Q. Agent와 RPA의 차이는?

> A. RPA는 정해진 UI 흐름을 반복 실행한다. Agent는 상황을 이해하고 유연하게 판단한다.
> RPA는 UI가 바뀌면 깨지지만, Agent는 목표 기반으로 대응한다.

Q. Pain이 여러 개면 Agent도 여러 개?

> A. 반드시 그렇지 않다. Pain들이 공통 Task로 묶이면 하나의 Agent로 처리 가능하다.
> 반대로 Pain이 하나여도 복잡하면 여러 Sub-agent로 분리할 수 있다.

🧪 Quiz — Agent에 적합한 시나리오는?

A) 사내 FAQ 문서에서 질문에 대한 답변 찾기

B) 고정된 HTML 템플릿에 데이터 채워 넣기

C) 이슈 티켓을 분석하고 담당자 지정 후 알림 발송

D) 주어진 텍스트를 영어에서 한국어로 번역

🧪 Quiz — Agent에 적합한 시나리오는?

A) 사내 FAQ 문서에서 질문에 대한 답변 찾기

B) 고정된 HTML 템플릿에 데이터 채워 넣기

C) 이슈 티켓을 분석하고 담당자 지정 후 알림 발송

D) 주어진 텍스트를 영어에서 한국어로 번역

정답: C

이슈 분석(LLM) + 담당자 DB 조회(Tool) + 알림 발송(Tool) → 다단계 추론 + 도구 사용

🧪 Quiz — Agent 패턴 분류

"사용자가 '경쟁사 동향 분석해줘'라고 입력하면, Agent가 검색 계획을 세우고 여러 소스를 탐색한 뒤 보고서를 작성한다."

이 시나리오는 자동화형 / 분석형 / Planner형 중 무엇인가?

🧪 Quiz — Agent 패턴 분류

"사용자가 '경쟁사 동향 분석해줘'라고 입력하면, Agent가 검색 계획을 세우고 여러 소스를 탐색한 뒤 보고서를 작성한다."

이 시나리오는 자동화형 / 분석형 / Planner형 중 무엇인가?

정답: Planner형

목표가 추상적이고, 실행 계획을 Agent가 결정하며, 탐색 결과에 따라 다음 행동이 달라진다

실습 — Lab 1 : Agent 문제 정의

- **시연** PydanticAI 기반 Agent 3가지 패턴 실행 (30분)
- **과제** 개인 업무 기반 Agent 후보 2-3개 도출 (45분)
- **공유** 발표 + 피드백 (15분)

labs/day1/s1-agent-problem-definition/ — README.md 참고

시연: PydanticAI 기본 — Agent 생성

```python
from pydantic_ai import Agent
from pydantic import BaseModel

# 가장 간단한 Agent — 모델만 지정
simple_agent = Agent("openai:gpt-5.4")
result = await simple_agent.run("PydanticAI를 한 줄로 설명해줘.")
print(result.output)

# instructions로 역할 부여
chef_agent = Agent(
    "openai:gpt-5.4",
    instructions="너는 친절한 요리사야. 모든 답변을 요리 비유로 해줘.",
)
```

시연: PydanticAI 기본 — Agent 생성

```python
from pydantic_ai import Agent
from pydantic import BaseModel

# 가장 간단한 Agent — 모델만 지정
simple_agent = Agent("openai:gpt-5.4")
result = await simple_agent.run("PydanticAI를 한 줄로 설명해줘.")
print(result.output)

# instructions로 역할 부여
chef_agent = Agent(
    "openai:gpt-5.4",
    instructions="너는 친절한 요리사야. 모든 답변을 요리 비유로 해줘.",
)
```

시연: PydanticAI 기본 — Agent 생성

```python
from pydantic_ai import Agent
from pydantic import BaseModel

# 가장 간단한 Agent — 모델만 지정
simple_agent = Agent("openai:gpt-5.4")
result = await simple_agent.run("PydanticAI를 한 줄로 설명해줘.")
print(result.output)

# instructions로 역할 부여
chef_agent = Agent(
    "openai:gpt-5.4",
    instructions="너는 친절한 요리사야. 모든 답변을 요리 비유로 해줘.",
)
```

시연: PydanticAI 기본 — Agent 생성

```python
from pydantic_ai import Agent
from pydantic import BaseModel

# 가장 간단한 Agent — 모델만 지정
simple_agent = Agent("openai:gpt-5.4")
result = await simple_agent.run("PydanticAI를 한 줄로 설명해줘.")
print(result.output)

# instructions로 역할 부여
chef_agent = Agent(
    "openai:gpt-5.4",
    instructions="너는 친절한 요리사야. 모든 답변을 요리 비유로 해줘.",
)
```

Agent(모델명) → await agent.run(프롬프트) → result.output

시연: Structured Output — output_type

```python
class MovieReview(BaseModel):
    title: str       # 영화 제목
    score: int       # 평점 (1~10)
    pros: list[str]  # 장점 목록
    cons: list[str]  # 단점 목록

review_agent = Agent(
    "openai:gpt-5.4",
    output_type=MovieReview,  # ← Pydantic 모델로 응답 강제
    instructions="영화 평론가로서 평가. 한국어로 작성.",
)

result = await review_agent.run("인터스텔라를 평가해주세요.")
review = result.output          # MovieReview 타입!
print(f"제목: {review.title}")  # 속성으로 바로 접근
print(f"평점: {review.score}/10")
```

시연: Structured Output — output_type

```python
class MovieReview(BaseModel):
    title: str        # 영화 제목
    score: int        # 평점 (1~10)
    pros: list[str]   # 장점 목록
    cons: list[str]   # 단점 목록

review_agent = Agent(
    "openai:gpt-5.4",
    output_type=MovieReview,  # ← Pydantic 모델로 응답 강제
    instructions="영화 평론가로서 평가. 한국어로 작성.",
)

result = await review_agent.run("인터스텔라를 평가해주세요.")
review = result.output          # MovieReview 타입!
print(f"제목: {review.title}")  # 속성으로 바로 접근
print(f"평점: {review.score}/10")
```

시연: Structured Output — output_type

```python
class MovieReview(BaseModel):
    title: str        # 영화 제목
    score: int        # 평점 (1~10)
    pros: list[str]   # 장점 목록
    cons: list[str]   # 단점 목록

review_agent = Agent(
    "openai:gpt-5.4",
    output_type=MovieReview,  # ← Pydantic 모델로 응답 강제
    instructions="영화 평론가로서 평가. 한국어로 작성.",
)

result = await review_agent.run("인터스텔라를 평가해주세요.")
review = result.output          # MovieReview 타입!
print(f"제목: {review.title}")  # 속성으로 바로 접근
print(f"평점: {review.score}/10")
```

시연: Structured Output — output_type

```python
class MovieReview(BaseModel):
    title: str        # 영화 제목
    score: int        # 평점 (1~10)
    pros: list[str]   # 장점 목록
    cons: list[str]   # 단점 목록

review_agent = Agent(
    "openai:gpt-5.4",
    output_type=MovieReview,  # ← Pydantic 모델로 응답 강제
    instructions="영화 평론가로서 평가. 한국어로 작성.",
)

result = await review_agent.run("인터스텔라를 평가해주세요.")
review = result.output           # MovieReview 타입!
print(f"제목: {review.title}")  # 속성으로 바로 접근
print(f"평점: {review.score}/10")
```

output_type=모델 → LLM이 반드시 해당 스키마로 응답

시연: Tool 등록 — @agent.tool_plain

```python
calc_agent = Agent(
    "openai:gpt-5.4",
    instructions="계산이 필요하면 제공된 도구를 사용하라.",
)

@calc_agent.tool_plain
def add(a: int, b: int) -> int:
    """두 숫자를 더한다."""  # ← docstring이 LLM에게 전달
    return a + b

@calc_agent.tool_plain
def multiply(a: int, b: int) -> int:
    """두 숫자를 곱한다."""
    return a * b

result = await calc_agent.run("17+28을 구하고, 결과에 3을 곱해.")
print(result.output)  # LLM이 add→multiply 순서로 자동 호출
```

시연: Tool 등록 — @agent.tool_plain

```python
calc_agent = Agent(
    "openai:gpt-5.4",
    instructions="계산이 필요하면 제공된 도구를 사용하라.",
)

@calc_agent.tool_plain
def add(a: int, b: int) → int:
    """두 숫자를 더한다."""  # ← docstring이 LLM에게 전달
    return a + b

@calc_agent.tool_plain
def multiply(a: int, b: int) → int:
    """두 숫자를 곱한다."""
    return a * b

result = await calc_agent.run("17+28을 구하고, 결과에 3을 곱해.")
print(result.output)  # LLM이 add→multiply 순서로 자동 호출
```

시연: Tool 등록 — @agent.tool_plain

```python
calc_agent = Agent(
    "openai:gpt-5.4",
    instructions="계산이 필요하면 제공된 도구를 사용하라.",
)

@calc_agent.tool_plain
def add(a: int, b: int) -> int:
    """두 숫자를 더한다."""  # ← docstring이 LLM에게 전달
    return a + b

@calc_agent.tool_plain
def multiply(a: int, b: int) -> int:
    """두 숫자를 곱한다."""
    return a * b

result = await calc_agent.run("17+28을 구하고, 결과에 3을 곱해.")
print(result.output)  # LLM이 add→multiply 순서로 자동 호출
```

시연: Tool 등록 — @agent.tool_plain

```python
calc_agent = Agent(
    "openai:gpt-5.4",
    instructions="계산이 필요하면 제공된 도구를 사용하라.",
)

@calc_agent.tool_plain
def add(a: int, b: int) -> int:
    """두 숫자를 더한다."""  # ← docstring이 LLM에게 전달
    return a + b

@calc_agent.tool_plain
def multiply(a: int, b: int) -> int:
    """두 숫자를 곱한다."""
    return a * b

result = await calc_agent.run("17+28을 구하고, 결과에 3을 곱해.")
print(result.output)  # LLM이 add→multiply 순서로 자동 호출
```

@agent.tool_plain + docstring → LLM이 알아서 도구 선택·호출

시연: 자동화형 Agent — GitHub 이슈 보고

```python
class DailyReport(BaseModel):
    summary: str
    issue_count: int
    highlights: list[str]

agent = Agent(
    "openai:gpt-5.4",
    output_type=DailyReport,
    instructions="fetch_github_issues로 이슈 수집 → send_slack_message로 발송",
)

@agent.tool_plain
def fetch_github_issues(repo: str) → str:
    """GitHub 저장소의 최근 이슈 목록을 가져온다."""
    resp = httpx.get(f"https://api.github.com/repos/{repo}/issues", ... )
    return "\n".join([f"#{i['number']} {i['title']}" for i in resp.json()])
```

시연: 자동화형 Agent — GitHub 이슈 보고

```python
class DailyReport(BaseModel):
    summary: str
    issue_count: int
    highlights: list[str]

agent = Agent(
    "openai:gpt-5.4",
    output_type=DailyReport,
    instructions="fetch_github_issues로 이슈 수집 → send_slack_message로 발송",
)

@agent.tool_plain
def fetch_github_issues(repo: str) -> str:
    """GitHub 저장소의 최근 이슈 목록을 가져온다."""
    resp = httpx.get(f"https://api.github.com/repos/{repo}/issues", ... )
    return "\n".join([f"#{i['number']} {i['title']}" for i in resp.json()])
```

시연: 자동화형 Agent — GitHub 이슈 보고

```python
agent = Agent(
    "openai:gpt-5.4",
    output_type=DailyReport,
    instructions="fetch_github_issues로 이슈 수집 → send_slack_message로 발송",
)

@agent.tool_plain
def fetch_github_issues(repo: str) → str:
    """GitHub 저장소의 최근 이슈 목록을 가져온다."""
    resp = httpx.get(f"https://api.github.com/repos/{repo}/issues", ... )
    return "\n".join([f"#{i['number']} {i['title']}" for i in resp.json()])

@agent.tool_plain
def send_slack_message(channel: str, message: str) → str:
    """Slack 채널에 메시지를 발송한다."""
    httpx.post("https://slack.com/api/chat.postMessage", ... )
    return f"#{channel} 발송 완료"
```

PydanticAI 핵심 패턴 정리

개념	역할	코드
Agent	LLM을 감싸는 핵심 객체	`Agent("openai:gpt-5.4")`
instructions	역할·규칙 부여	`instructions="..."`
output_type	응답을 Pydantic 모델로 강제	`output_type=MyModel`
@tool_plain	LLM이 호출할 함수 등록	`@agent.tool_plain`
await run()	Agent 실행	`await agent.run("...")`

PydanticAI 핵심 패턴 정리

개념	역할	코드
Agent	LLM을 감싸는 핵심 객체	`Agent("openai:gpt-5.4")`
instructions	역할·규칙 부여	`instructions="..."`
output_type	응답을 Pydantic 모델로 강제	`output_type=MyModel`
@tool_plain	LLM이 호출할 함수 등록	`@agent.tool_plain`
await run()	Agent 실행	`await agent.run("...")`

이 패턴들이 모든 실습의 기반 — **Agent + instructions + output_type + Tool**

☕ 쉬는 시간

10분 후 2교시를 시작합니다

2교시

LLM 동작 원리 및 프롬프트 전략

Context Window · Hallucination · Structured Output

프롬프트는 Agent의 두뇌 설계다

프롬프트는 Agent의 두뇌 설계다

Agent를 만든다 = **LLM을 코드로 제어**한다

프롬프트는 Agent의 두뇌 설계다

Agent를 만든다 = **LLM을 코드로 제어**한다

LLM이 어떻게 작동하는지 모르면 Agent 동작을 **예측할 수 없다**

프롬프트는 Agent의 두뇌 설계다

Agent를 만든다 = **LLM을 코드로 제어**한다

LLM이 어떻게 작동하는지 모르면 Agent 동작을 **예측할 수 없다**

프롬프트 전략이 **결과 품질**과 **비용**을 동시에 결정한다

프롬프트는 Agent의 두뇌 설계다

Agent를 만든다 = **LLM을 코드로 제어**한다

LLM이 어떻게 작동하는지 모르면 Agent 동작을 **예측할 수 없다**

프롬프트 전략이 **결과 품질**과 **비용**을 동시에 결정한다

올바른 프롬프트 전략은 **비용 50%**, **지연 30%**를 줄일 수 있다

Token과 Context Window

Token
단어/글자 조각

⊂

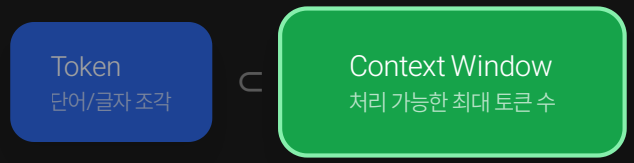
Context Window
처리 가능한 최대 토큰 수

클릭하여 확인

Token과 Context Window

Token: 영어 1단어 ≈ 1-2토큰 · 한국어 1글자 ≈ 1-2토큰 · 코드 100줄 ≈ 500-1,000토큰

Token과 Context Window



Context Window = System Prompt + 대화 이력 + 현재 입력 + 출력 전부 포함

Context Window 구조

= 총 3,800 토큰 사용

Context Window 구조

= 총 3,800 토큰 사용

- 긴 대화 이력은 Context를 **빠르게 소모**한다

Context Window 구조

= 총 3,800 토큰 사용

- 긴 대화 이력은 Context를 **빠르게 소모**한다
- Context 초과 시 **앞부분이 잘리거나 에러 발생**

Context Window 구조

= 총 3,800 토큰 사용

- 긴 대화 이력은 Context를 **빠르게 소모**한다
- Context 초과 시 **앞부분이 잘리거나 에러 발생**
- RAG의 검색 결과도 Context를 소모한다

Hallucination — LLM이 거짓을 자신 있게 말한다

확률 기반 모델의 **구조적 한계**

Hallucination — LLM이 거짓을 자신 있게 말한다

확률 기반 모델의 **구조적 한계**

① 최신 정보가 필요한 질문 (학습 데이터 이후 사건)

Hallucination — LLM이 거짓을 자신 있게 말한다

확률 기반 모델의 **구조적 한계**

① 최신 정보가 필요한 질문 (학습 데이터 이후 사건)

② 구체적 수치, 날짜, 인용 요청

Hallucination — LLM이 거짓을 자신 있게 말한다

확률 기반 모델의 **구조적 한계**

① 최신 정보가 필요한 질문 (학습 데이터 이후 사건)

② 구체적 수치, 날짜, 인용 요청

③ 너무 제약이 없는 열린 질문

Hallucination — LLM이 거짓을 자신 있게 말한다

확률 기반 모델의 **구조적 한계**

① 최신 정보가 필요한 질문 (학습 데이터 이후 사건)

② 구체적 수치, 날짜, 인용 요청

③ 너무 제약이 없는 열린 질문

④ 모델이 잘 모르는 도메인의 세부 사항

Hallucination — LLM이 거짓을 자신 있게 말한다

확률 기반 모델의 **구조적 한계**

① 최신 정보가 필요한 질문 (학습 데이터 이후 사건)

② 구체적 수치, 날짜, 인용 요청

③ 너무 제약이 없는 열린 질문

④ 모델이 잘 모르는 도메인의 세부 사항

Agent 설계 시 "**LLM이 틀릴 수 있다**"는 전제로 검증 로직을 포함하라

Hallucination 완화 전략

전략	설명	방법
근거 요청	"왜 그렇게 생각하는지 설명하라"	CoT 프롬프트
출처 제공	관련 문서를 Context에 포함	RAG 결합
불확실성 명시	"모르면 모른다고 답하라"	System Prompt
검증 루프	생성 후 별도 검증 단계	자기 검토
구조화 출력	JSON Schema로 응답 제한	Structured Output

Hallucination 완화 전략

전략	설명	방법
근거 요청	"왜 그렇게 생각하는지 설명하라"	CoT 프롬프트
출처 제공	관련 문서를 Context에 포함	RAG 결합
불확실성 명시	"모르면 모른다고 답하라"	System Prompt
검증 루프	생성 후 별도 검증 단계	자기 검토
구조화 출력	JSON Schema로 응답 제한	Structured Output

RAG + Structured Output + 검증 루프 = Hallucination 대폭 감소

Zero-shot 전략

지시만 제공하고 **예시 없이** 수행 요청

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

Zero-shot 전략

지시만 제공하고 **예시 없이** 수행 요청

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

장점

프롬프트 짧음

토큰 효율

Zero-shot 전략

지시만 제공하고 **예시 없이** 수행 요청

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

장점
프롬프트 짧음
토큰 효율

단점
복잡한 작업에서
품질 저하

Zero-shot 전략

지시만 제공하고 **예시 없이** 수행 요청

```
prompt = """
다음 고객 리뷰를 긍정/부정/중립으로 분류하라.

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
"""
```

장점
프롬프트 짧음
토큰 효율

단점
복잡한 작업에서
품질 저하

적합
단순 분류
번역, 요약

Few-shot 전략

작업 방식을 **예시(shot)**로 보여준다

```
prompt = """
고객 리뷰를 긍정/부정/중립으로 분류하라.

예시 1:
리뷰: "정말 마음에 들어요!" → 긍정

예시 2:
리뷰: "완전 실망입니다." → 부정

예시 3:
리뷰: "평범한 제품이에요." → 중립

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
분류:
```

장점: 출력 형식과 품질 안정적

단점: 토큰 사용량 증가

Few-shot 전략

작업 방식을 **예시(shot)**로 보여준다

```
고객 리뷰를 긍정/부정/중립으로 분류하라.

예시 1:
리뷰: "정말 마음에 들어요!" → 긍정

예시 2:
리뷰: "완전 실망입니다." → 부정

예시 3:
리뷰: "평범한 제품이에요." → 중립

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
분류:
"""
```

장점: 출력 형식과 품질 안정적

단점: 토큰 사용량 증가

Few-shot 전략

작업 방식을 **예시(shot)**로 보여준다

```
prompt = """
고객 리뷰를 긍정/부정/중립으로 분류하라.

예시 1:
리뷰: "정말 마음에 들어요!" → 긍정

예시 2:
리뷰: "완전 실망입니다." → 부정

예시 3:
리뷰: "평범한 제품이에요." → 중립

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."
분류:
```

장점: 출력 형식과 품질 안정적

단점: 토큰 사용량 증가

Chain-of-Thought (CoT) 전략

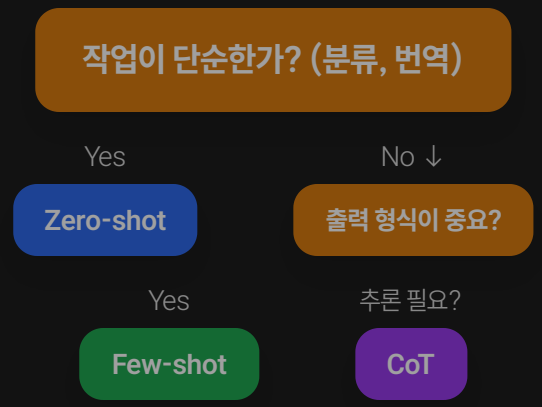
단계적 추론 과정을 포함하도록 유도

```
prompt = """
다음 리뷰를 단계별로 분석하여 분류하라.

리뷰: "배송이 빠르긴 했는데 포장이 엉망이었어요."

1단계: 긍정적 표현 추출
2단계: 부정적 표현 추출
3단계: 전체 감정 판단
4단계: 최종 분류
"""
```

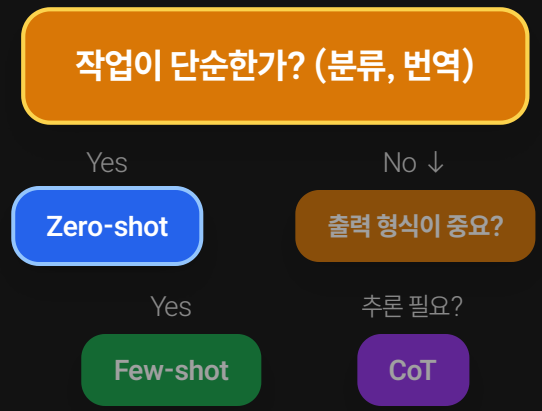
장점: 복잡한 추론 정확도 **대폭 향상**

단점: 출력 토큰 증가 → 비용/지연 증가

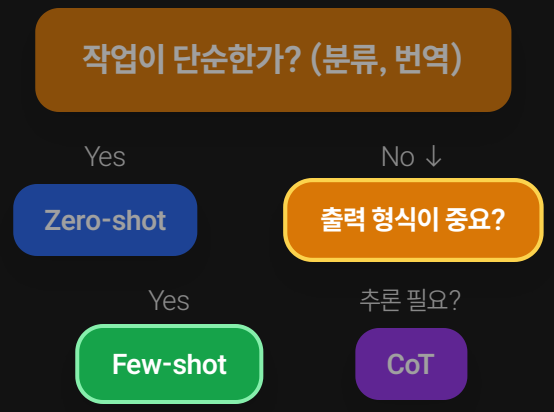
전략 선택 가이드



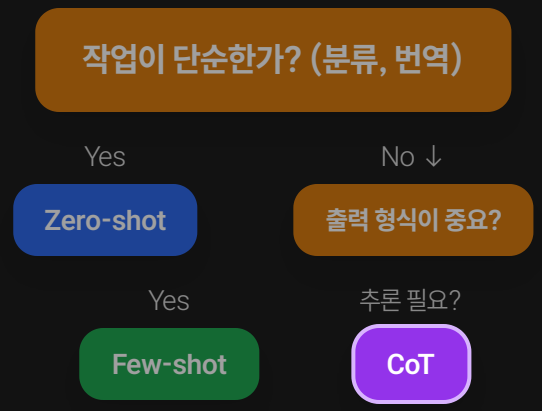
전략 선택 가이드



전략 선택 가이드



전략 선택 가이드



전략별 특성 비교

전략	토큰 사용	정확도	응답 속도	추천 상황
Zero-shot	낮음	보통	빠름	단순 작업
Few-shot	중간	높음	중간	형식 중요
CoT	높음	높음	느림	복잡한 추론
CoT + Few-shot	매우 높음	**매우 높음**	느림	고정확도

전략별 특성 비교

전략	토큰 사용	정확도	응답 속도	추천 상황
Zero-shot	낮음	보통	빠름	단순 작업
Few-shot	중간	높음	중간	형식 중요
CoT	높음	높음	느림	복잡한 추론
CoT + Few-shot	매우 높음	**매우 높음**	느림	고정확도

비용 < 정확도 → Zero-shot 먼저 · **정확도 > 비용** → Few-shot + 검증

Structured Output — JSON Schema

입력
"리뷰 분류해줘"

→

LLM
JSON Schema 강제

→

출력
{"category": "부정"}

클릭하여 확인

Structured Output — JSON Schema

입력
"리뷰 분류해줘"

→

LLM
JSON Schema 강제

→

출력
{"category": "부정"}

Agent에서 LLM 출력은 다음 로직의**입력**으로 사용된다

Structured Output — JSON Schema

입력
"리뷰 분류해줘"

→

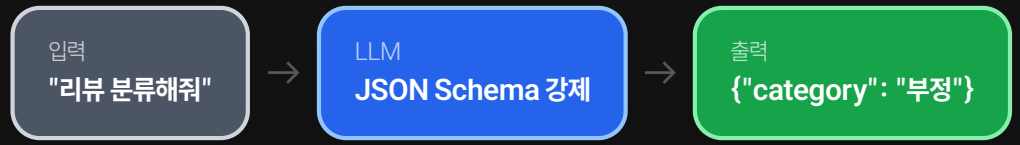
LLM
JSON Schema 강제

→

출력
{"category": "부정"}

자연어 파싱은불안정— JSON Schema로 응답 형식을강제

Structured Output — JSON Schema



Tool Use 방식이 더 안정적 — LLM이**반드시**지정한 스키마로 응답

PydanticAI의 Structured Output

```python
from pydantic import BaseModel, Field
from enum import Enum

class Sentiment(str, Enum):
    positive = "긍정"
    negative = "부정"
    neutral = "중립"

class ReviewResult(BaseModel):
    category: Sentiment
    confidence: float = Field(ge=0, le=1)
    reason: str = Field(description="분류 이유 (1문장)")

from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", output_type=ReviewResult)
result = await agent.run("배송이 빠르긴 했는데 포장이 엉망이었어요.")
print(result.output.category)    # Sentiment.negative
print(result.output.confidence)  # 0.85
```

PydanticAI의 Structured Output

```python
from pydantic import BaseModel, Field
from enum import Enum

class Sentiment(str, Enum):
    positive = "긍정"
    negative = "부정"
    neutral = "중립"

class ReviewResult(BaseModel):
    category: Sentiment
    confidence: float = Field(ge=0, le=1)
    reason: str = Field(description="분류 이유 (1문장)")

from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", output_type=ReviewResult)
result = await agent.run("배송이 빠르긴 했는데 포장이 엉망이었어요.")
print(result.output.category)    # Sentiment.negative
print(result.output.confidence)  # 0.85
```

PydanticAI의 Structured Output

```python
from pydantic import BaseModel, Field
from enum import Enum

class Sentiment(str, Enum):
    positive = "긍정"
    negative = "부정"
    neutral = "중립"

class ReviewResult(BaseModel):
    category: Sentiment
    confidence: float = Field(ge=0, le=1)
    reason: str = Field(description="분류 이유 (1문장)")

from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", output_type=ReviewResult)
result = await agent.run("배송이 빠르긴 했는데 포장이 엉망이었어요.")
print(result.output.category)    # Sentiment.negative
print(result.output.confidence)  # 0.85
```

PydanticAI의 Structured Output

```python
from pydantic import BaseModel, Field
from enum import Enum

class Sentiment(str, Enum):
    positive = "긍정"
    negative = "부정"
    neutral = "중립"

class ReviewResult(BaseModel):
    category: Sentiment
    confidence: float = Field(ge=0, le=1)
    reason: str = Field(description="분류 이유 (1문장)")

from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", output_type=ReviewResult)
result = await agent.run("배송이 빠르긴 했는데 포장이 엉망이었어요.")
print(result.output.category)    # Sentiment.negative
print(result.output.confidence)  # 0.85
```

output_type으로 Pydantic 모델 지정 → 파싱 실패율 대폭 감소

비용 계산과 최적화

비용 = (입력 토큰 × 입력 단가) + (출력 토큰 × 출력 단가)

비용 계산과 최적화

비용 = (입력 토큰 × 입력 단가) + (출력 토큰 × 출력 단가)

1순위 **모델 선택 최적화** — 단순 분류에 고성능 모델 불필요

비용 계산과 최적화

비용 = (입력 토큰 × 입력 단가) + (출력 토큰 × 출력 단가)

1순위 **모델 선택 최적화** — 단순 분류에 고성능 모델 불필요

2순위 **출력 길이 제한** — max_tokens 적절히 설정

비용 계산과 최적화

비용 = (입력 토큰 × 입력 단가) + (출력 토큰 × 출력 단가)

1순위 **모델 선택 최적화** — 단순 분류에 고성능 모델 불필요

2순위 **출력 길이 제한** — max_tokens 적절히 설정

3순위 **입력 압축** — 불필요한 Context 제거

비용 계산과 최적화

비용 = (입력 토큰 × 입력 단가) + (출력 토큰 × 출력 단가)

- 1순위 **모델 선택 최적화** — 단순 분류에 고성능 모델 불필요
- 2순위 **출력 길이 제한** — max_tokens 적절히 설정
- 3순위 **입력 압축** — 불필요한 Context 제거
- 4순위 **캐싱 활용** — 반복 System Prompt 캐싱

Latency 최적화

병렬 호출

3개 리뷰 입력

↓ 동시에

LLM ① LLM ② LLM ③

↓ 취합

결과 3개

스트리밍

요청

↓ 실시간

응답 스트리밍
토큰 단위로 전달

↓

체감 속도 개선

프롬프트 설계 실수 5가지

프롬프트 설계 실수 5가지

① **System Prompt 비대화** 매 호출마다 비용 증가

프롬프트 설계 실수 5가지

① **System Prompt 비대화** 매 호출마다 비용 증가

② **JSON 파싱 실패 처리 누락** 항상 try-except 필수

프롬프트 설계 실수 5가지

① **System Prompt 비대화** 매 호출마다 비용 증가

② **JSON 파싱 실패 처리 누락** 항상 try-except 필수

③ **max_tokens 과소 설정** CoT 응답은 예상보다 길다

프롬프트 설계 실수 5가지

1. **System Prompt 비대화** 매 호출마다 비용 증가
2. **JSON 파싱 실패 처리 누락** 항상 try-except 필수
3. **max_tokens 과소 설정** CoT 응답은 예상보다 길다
4. **모델 과사용** 단계별로 적합한 모델 선택

프롬프트 설계 실수 5가지

1. **System Prompt 비대화** 매 호출마다 비용 증가
2. **JSON 파싱 실패 처리 누락** 항상 try-except 필수
3. **max_tokens 과소 설정** CoT 응답은 예상보다 길다
4. **모델 과사용** 단계별로 적합한 모델 선택
5. **프롬프트 인젝션 무시** 사용자 입력을 별도 태그로 감싸라

❓ Q&A — Session 2

Q. Context Window가 길수록 무조건 좋은가?

❓ Q&A — Session 2

Q. Context Window가 길수록 무조건 좋은가?

> A. 더 많은 정보를 처리할 수 있지만 비용도 증가한다.
> Context가 길어지면 중간 정보를 놓치는 "lost in the middle" 현상이 발생한다.

❓ Q&A — Session 2

Q. Context Window가 길수록 무조건 좋은가?

> A. 더 많은 정보를 처리할 수 있지만 비용도 증가한다.
> Context가 길어지면 중간 정보를 놓치는 "lost in the middle" 현상이 발생한다.

❓ Q&A — Session 2

Q. Context Window가 길수록 무조건 좋은가?

> A. 더 많은 정보를 처리할 수 있지만 비용도 증가한다.
> Context가 길어지면 중간 정보를 놓치는 "lost in the middle" 현상이 발생한다.

Q. Hallucination을 완전히 없앨 수 있는가?

> A. 현재 기술로는 완전히 제거하기 어렵다.
> RAG + Structured Output + 검증 단계를 추가하면 대폭 줄일 수 있다.

🧪 Quiz — Context Window

Agent 실행 중 "context length exceeded" 에러 발생 시 가장 먼저 확인할 것은?

A) LLM 모델 버전

B) System Prompt + 대화 이력 + 현재 입력의 총 토큰 수

C) 인터넷 연결 상태

D) API 키 유효성

🧪 Quiz — Context Window

Agent 실행 중 "context length exceeded" 에러 발생 시 가장 먼저 확인할 것은?

A) LLM 모델 버전

B) System Prompt + 대화 이력 + 현재 입력의 총 토큰 수

C) 인터넷 연결 상태

D) API 키 유효성

정답: B

Context Window 초과는 입력 토큰 총합이 한계를 넘었을 때 발생한다

🧪 Quiz — 프롬프트 전략 선택

"보험 청구서 PDF를 분석하여 청구 유효성을 판단. 판단 근거를 단계별로 설명해야 하며, 실수 허용 범위가 매우 낮다."

가장 적합한 프롬프트 전략은?

🧪 Quiz — 프롬프트 전략 선택

"보험 청구서 PDF를 분석하여 청구 유효성을 판단. 판단 근거를 단계별로 설명해야 하며, 실수 허용 범위가 매우 낮다."

가장 적합한 프롬프트 전략은?

정답: CoT + Few-shot 조합

CoT: 단계별 추론 → 정확도 향상 · Few-shot: 올바른 판단 예시 → 일관성 확보
+ Structured Output 추가 시 더욱 안정적

실습 — Lab 2: Context 전략 비교

- **시연** 컨텍스트 전략별 응답 비교 (20분)
- **실습** LangSmith 셋업 + Structured Output (25분)
- **과제** 자기 도메인에 전략 적용 + 비교 분석 (35분)

labs/day1/s2-context-strategy/ — README.md 참고

시연: System Prompt 변형 비교

```
# 전략 A: System Prompt 없음 — 형식 불안정
agent_a = Agent(MODEL)

# 전략 B: 역할 지정 — 품질 향상, 형식은 자유
agent_b = Agent(MODEL,
    instructions="이커머스 고객 경험 분석 전문가입니다.",
)

# 전략 C: 역할 + 제약 + 출력형식 — 가장 안정
agent_c = Agent(MODEL,
    instructions=(
        "이커머스 고객 경험 분석 전문가.\n"
        "규칙:\n"
        "1. 반드시 JSON만 출력. 다른 텍스트 금지.\n"
        "2. 감성은 '긍정/부정/중립' 중 하나.\n"
        "3. 키워드 최대 3개.\n"
        "4. 요약 1문장 20자 이내."
    ),
)
```

시연: System Prompt 변형 비교

```
# 전략 A: System Prompt 없음 — 형식 불안정
agent_a = Agent(MODEL)

# 전략 B: 역할 지정 — 품질 향상, 형식은 자유
agent_b = Agent(MODEL,
    instructions="이커머스 고객 경험 분석 전문가입니다.",
)

# 전략 C: 역할 + 제약 + 출력형식 — 가장 안정
agent_c = Agent(MODEL,
    instructions=(
        "이커머스 고객 경험 분석 전문가.\n"
        "규칙:\n"
        "1. 반드시 JSON만 출력. 다른 텍스트 금지.\n"
        "2. 감성은 '긍정/부정/중립' 중 하나.\n"
        "3. 키워드 최대 3개.\n"
        "4. 요약 1문장 20자 이내."
    ),
)
```

시연: System Prompt 변형 비교

```
# 전략 A: System Prompt 없음 — 형식 불안정
agent_a = Agent(MODEL)

# 전략 B: 역할 지정 — 품질 향상, 형식은 자유
agent_b = Agent(MODEL,
    instructions="이커머스 고객 경험 분석 전문가입니다.",
)

# 전략 C: 역할 + 제약 + 출력형식 — 가장 안정
agent_c = Agent(MODEL,
    instructions=(
        "이커머스 고객 경험 분석 전문가.\n"
        "규칙:\n"
        "1. 반드시 JSON만 출력. 다른 텍스트 금지.\n"
        "2. 감성은 '긍정/부정/중립' 중 하나.\n"
        "3. 키워드 최대 3개.\n"
        "4. 요약 1문장 20자 이내."
    ),
)
```

시연: System Prompt 변형 비교

```
# 전략 A: System Prompt 없음 — 형식 불안정
agent_a = Agent(MODEL)

# 전략 B: 역할 지정 — 품질 향상, 형식은 자유
agent_b = Agent(MODEL,
    instructions="이커머스 고객 경험 분석 전문가입니다.",
)

# 전략 C: 역할 + 제약 + 출력형식 — 가장 안정
agent_c = Agent(MODEL,
    instructions=(
        "이커머스 고객 경험 분석 전문가.\n"
        "규칙:\n"
        "1. 반드시 JSON만 출력. 다른 텍스트 금지.\n"
        "2. 감성은 '긍정/부정/중립' 중 하나.\n"
        "3. 키워드 최대 3개.\n"
        "4. 요약 1문장 20자 이내."
    ),
)
```

instructions가 길어질수록 토큰 증가 but **형식 안정성 향상**

시연: Structured Output vs 자유 응답

```python
# Pydantic 모델로 스키마 정의
class ReviewAnalysis(BaseModel):
    sentiment: str = Field(description="감성: 긍정/부정/중립")
    keywords: list[str] = Field(description="핵심 키워드 (최대 3개)")
    summary: str = Field(description="리뷰 요약 (1문장)")
    confidence: float = Field(ge=0, le=1)

# 자유 응답 — 형식이 매번 달라짐
agent_text = Agent(MODEL, instructions="리뷰를 분석해주세요.")

# Structured Output — 100% 스키마 보장
agent_struct = Agent(MODEL,
    output_type=ReviewAnalysis,  # ← 핵심
    instructions="리뷰를 분석하세요.",
)
result = await agent_struct.run(review)
r = result.output              # ReviewAnalysis 타입
print(r.sentiment, r.keywords) # 타입 안전 접근
```

시연: Structured Output vs 자유 응답

```python
# Pydantic 모델로 스키마 정의
class ReviewAnalysis(BaseModel):
    sentiment: str = Field(description="감성: 긍정/부정/중립")
    keywords: list[str] = Field(description="핵심 키워드 (최대 3개)")
    summary: str = Field(description="리뷰 요약 (1문장)")
    confidence: float = Field(ge=0, le=1)

# 자유 응답 — 형식이 매번 달라짐
agent_text = Agent(MODEL, instructions="리뷰를 분석해주세요.")

# Structured Output — 100% 스키마 보장
agent_struct = Agent(MODEL,
    output_type=ReviewAnalysis,  # ← 핵심
    instructions="리뷰를 분석하세요.",
)
result = await agent_struct.run(review)
r = result.output              # ReviewAnalysis 타입
print(r.sentiment, r.keywords) # 타입 안전 접근
```

시연: Structured Output vs 자유 응답

```python
# Pydantic 모델로 스키마 정의
class ReviewAnalysis(BaseModel):
    sentiment: str = Field(description="감성: 긍정/부정/중립")
    keywords: list[str] = Field(description="핵심 키워드 (최대 3개)")
    summary: str = Field(description="리뷰 요약 (1문장)")
    confidence: float = Field(ge=0, le=1)

# 자유 응답 — 형식이 매번 달라짐
agent_text = Agent(MODEL, instructions="리뷰를 분석해주세요.")

# Structured Output — 100% 스키마 보장
agent_struct = Agent(MODEL,
    output_type=ReviewAnalysis,  # ← 핵심
    instructions="리뷰를 분석하세요.",
)
result = await agent_struct.run(review)
r = result.output              # ReviewAnalysis 타입
print(r.sentiment, r.keywords) # 타입 안전 접근
```

시연: Structured Output vs 자유 응답

```python
# Pydantic 모델로 스키마 정의
class ReviewAnalysis(BaseModel):
    sentiment: str = Field(description="감성: 긍정/부정/중립")
    keywords: list[str] = Field(description="핵심 키워드 (최대 3개)")
    summary: str = Field(description="리뷰 요약 (1문장)")
    confidence: float = Field(ge=0, le=1)

# 자유 응답 — 형식이 매번 달라짐
agent_text = Agent(MODEL, instructions="리뷰를 분석해주세요.")

# Structured Output — 100% 스키마 보장
agent_struct = Agent(MODEL,
    output_type=ReviewAnalysis,  # ← 핵심
    instructions="리뷰를 분석하세요.",
)
result = await agent_struct.run(review)
r = result.output              # ReviewAnalysis 타입
print(r.sentiment, r.keywords) # 타입 안전 접근
```

Agent 파이프라인에서는 **Structured Output이 필수** — 다음 단계 입력으로 바로 사용

시연: 비용·토큰 측정

```python
async def run_agent(agent, prompt, label) -> dict:
    start = time.time()
    result = await agent.run(prompt)
    usage = result.usage()  # 입력/출력 토큰 수 조회
    return {
        "input_tokens": usage.input_tokens,
        "output_tokens": usage.output_tokens,
        "latency_sec": round(time.time() - start, 2),
        "cost_usd": calculate_cost(MODEL, usage.input_tokens, usage.output_tokens),
    }

# 비용 계산: 단가 × 토큰 수
# 출력 토큰이 입력보다 ~4배 비싸다!
PRICING = {
    "openai:gpt-5.4":      {"input": 2.50, "output": 10.00},  # $/1M tokens
    "openai:gpt-5.4-mini":  {"input": 0.15, "output": 0.60},
}
```

시연: 비용·토큰 측정

```python
async def run_agent(agent, prompt, label) -> dict:
    start = time.time()
    result = await agent.run(prompt)
    usage = result.usage()  # 입력/출력 토큰 수 조회
    return {
        "input_tokens": usage.input_tokens,
        "output_tokens": usage.output_tokens,
        "latency_sec": round(time.time() - start, 2),
        "cost_usd": calculate_cost(MODEL, usage.input_tokens, usage.output_tokens),
    }

# 비용 계산: 단가 × 토큰 수
# 출력 토큰이 입력보다 ~4배 비싸다!
PRICING = {
    "openai:gpt-5.4":      {"input": 2.50, "output": 10.00},  # $/1M tokens
    "openai:gpt-5.4-mini":  {"input": 0.15, "output": 0.60},
}
```

시연: 비용·토큰 측정

```python
async def run_agent(agent, prompt, label) -> dict:
    start = time.time()
    result = await agent.run(prompt)
    usage = result.usage()  # 입력/출력 토큰 수 조회
    return {
        "input_tokens": usage.input_tokens,
        "output_tokens": usage.output_tokens,
        "latency_sec": round(time.time() - start, 2),
        "cost_usd": calculate_cost(MODEL, usage.input_tokens, usage.output_tokens),
    }

# 비용 계산: 단가 × 토큰 수
# 출력 토큰이 입력보다 ~4배 비싸다!
PRICING = {
    "openai:gpt-5.4":      {"input": 2.50, "output": 10.00},  # $/1M tokens
    "openai:gpt-5.4-mini":  {"input": 0.15, "output": 0.60},
}
```

result.usage()로 토큰 측정 → 전략별 비용 비교가 핵심

🍽️ 점심시간

13:00 — 14:00

3교시

Agent 기획서 구조화

Task 분해 · Stateless vs Stateful · IPO 명세 · 실패 처리

기획 없는 Agent는 방향 없는 자동화다

기획 없는 Agent는 방향 없는 자동화다

코드를 먼저 짜고 싶은 충동이 있다

기획 없는 Agent는 방향 없는 자동화다

코드를 먼저 짜고 싶은 충동이 있다

명확하지 않은 Agent는 개발 중반에 **방향을 잃는다**

기획 없는 Agent는 방향 없는 자동화다

코드를 먼저 짜고 싶은 충동이 있다

명확하지 않은 Agent는 개발 중반에 **방향을 잃는다**

구조화된 기획서가 개발 속도와 품질을 동시에 높인다

기획 없는 Agent는 방향 없는 자동화다

코드를 먼저 짜고 싶은 충동이 있다

명확하지 않은 Agent는 개발 중반에 **방향을 잃는다**

구조화된 기획서가 개발 속도와 품질을 동시에 높인다

"어디서 시작해서 어디서 끝나는지" 모르면 테스트도, 예외 처리도 할 수 없다

Task → Sub-task → Workflow 분해

Task → Sub-task → Workflow 분해

- 하나의 Sub-task는 **하나의 책임**만 가진다

Task → Sub-task → Workflow 분해

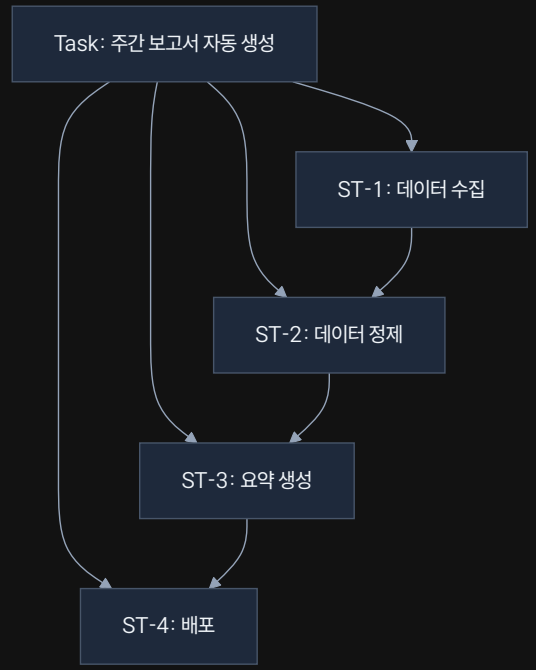
- 하나의 Sub-task는 **하나의 책임**만 가진다
- Sub-task는 독립적으로 **테스트 가능**해야 한다

Task → Sub-task → Workflow 분해

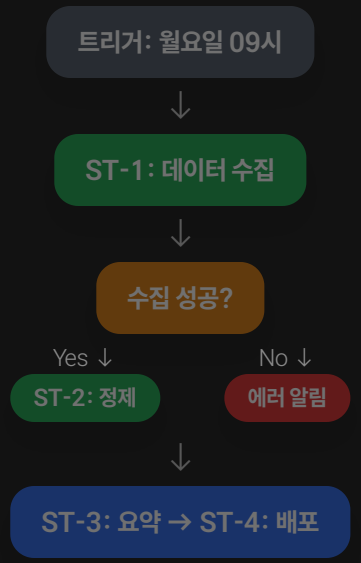
- 하나의 Sub-task는 **하나의 책임**만 가진다
- Sub-task는 독립적으로 **테스트 가능**해야 한다
- Sub-task 출력이 다음 Sub-task **입력**이 된다

Task → Sub-task → Workflow 분해

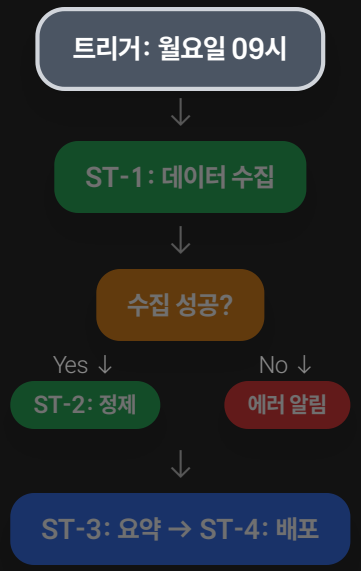
- 하나의 Sub-task는 **하나의 책임**만 가진다
- Sub-task는 독립적으로 **테스트 가능**해야 한다
- Sub-task 출력이 다음 Sub-task **입력**이 된다



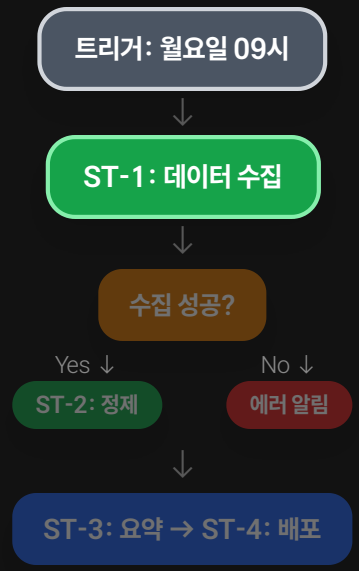
Workflow 다이어그램



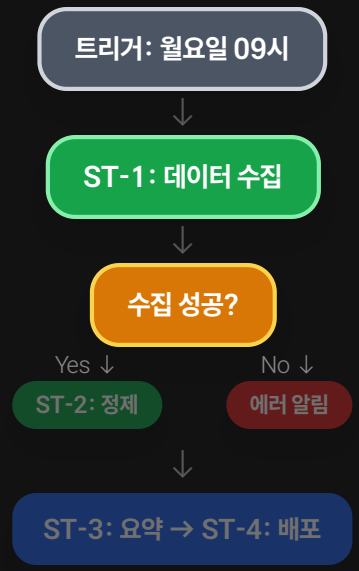
Workflow 다이어그램



Workflow 다이어그램

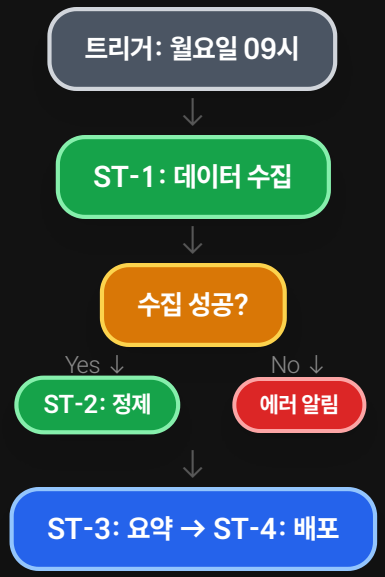


Workflow 다이어그램



Workflow 다이어그램

Workflow 다이어그램



Stateless vs Stateful

Stateless

Stateful

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**

Stateful

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**

Stateful

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**

Stateful

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**
- 예: 티켓 분류, 번역

Stateful

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**
- 예: 티켓 분류, 번역

Stateful

- 이전 결과를 **기억**

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**
- 예: 티켓 분류, 번역

Stateful

- 이전 결과를 **기억**
- 단계적 처리 **가능**

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**
- 예: 티켓 분류, 번역

Stateful

- 이전 결과를 **기억**
- 단계적 처리 **가능**
- 대화형 상호작용 **지원**

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**
- 예: 티켓 분류, 번역

Stateful

- 이전 결과를 **기억**
- 단계적 처리 **가능**
- 대화형 상호작용 **지원**
- 예: 리서치, Assistant

Stateless vs Stateful

Stateless

- 각 호출이 **독립적**
- 수평 확장 **쉬움**
- 실패 시 재시도 **안전**
- 예: 티켓 분류, 번역

Stateful

- 이전 결과를 **기억**
- 단계적 처리 **가능**
- 대화형 상호작용 **지원**
- 예: 리서치, Assistant

이전 실행 결과가 다음 실행에 필요한가? → **Yes: Stateful · No: Stateless**

Stateless vs Stateful 비교

항목	Stateless	Stateful
구현 복잡도	낮음	**높음**
확장성	**매우 좋음**	제한적
실패 복구	단순 재시도	체크포인트 필요
메모리 사용	낮음	**높음**
적합 패턴	자동화형	**Planner형**
디버깅	쉬움	**어려움**

IPO 명세 — Input · Process · Output

모든 Sub-task는 **I/O가 명확**해야 한다 — 명확하지 않으면 테스트할 수 없다

```
# INPUT
@dataclass
class TicketInput:
    ticket_id: str
    content: str
    priority: Literal["low", "medium", "high"] = "medium"

# OUTPUT
@dataclass
class TicketOutput:
    category: Literal["bug", "feature", "question"]
    severity: int  # 1-5
    assignee: str
    reason: str

# EXCEPTION
```

IPO 명세 — Input · Process · Output

모든 Sub-task는 **I/O가 명확**해야 한다 — 명확하지 않으면 테스트할 수 없다

```python
class TicketInput:
    ticket_id: str
    content: str
    priority: Literal["low", "medium", "high"] = "medium"

# OUTPUT
@dataclass
class TicketOutput:
    category: Literal["bug", "feature", "question"]
    severity: int  # 1-5
    assignee: str
    reason: str

# EXCEPTION
# content 빈 값 → ValueError
# LLM 응답 파싱 실패 → category="unclassified"
```

IPO 명세 — Input · Process · Output

모든 Sub-task는 **I/O가 명확**해야 한다 — 명확하지 않으면 테스트할 수 없다

```
class TicketInput:
    ticket_id: str
    content: str
    priority: Literal["low", "medium", "high"] = "medium"

# OUTPUT
@dataclass
class TicketOutput:
    category: Literal["bug", "feature", "question"]
    severity: int  # 1-5
    assignee: str
    reason: str

# EXCEPTION
# content 빈 값 → ValueError
# LLM 응답 파싱 실패 → category="unclassified"
```

실패 유형 분류

유형	예시	처리 전략

실패 유형 분류

유형	예시	처리 전략
외부 API 실패	Jira 타임아웃	재시도 + 대체 데이터

실패 유형 분류

유형	예시	처리 전략
외부 API 실패	Jira 타임아웃	재시도 + 대체 데이터
LLM 응답 오류	JSON 파싱 실패	재시도 + 기본값

실패 유형 분류

유형	예시	처리 전략
외부 API 실패	Jira 타임아웃	재시도 + 대체 데이터
LLM 응답 오류	JSON 파싱 실패	재시도 + 기본값
데이터 오류	필수 필드 누락	검증 후 조기 실패

실패 유형 분류

유형	예시	처리 전략
외부 API 실패	Jira 타임아웃	재시도 + 대체 데이터
LLM 응답 오류	JSON 파싱 실패	재시도 + 기본값
데이터 오류	필수 필드 누락	검증 후 조기 실패
비즈니스 오류	예산 초과	사람에게 에스컬레이션

실패 유형 분류

유형	예시	처리 전략
외부 API 실패	Jira 타임아웃	재시도 + 대체 데이터
LLM 응답 오류	JSON 파싱 실패	재시도 + 기본값
데이터 오류	필수 필드 누락	검증 후 조기 실패
비즈니스 오류	예산 초과	사람에게 에스컬레이션
무한 루프	같은 행동 반복	최대 시도 횟수 제한

실패 유형 분류

유형	예시	처리 전략
외부 API 실패	Jira 타임아웃	재시도 + 대체 데이터
LLM 응답 오류	JSON 파싱 실패	재시도 + 기본값
데이터 오류	필수 필드 누락	검증 후 조기 실패
비즈니스 오류	예산 초과	사람에게 에스컬레이션
무한 루프	같은 행동 반복	최대 시도 횟수 제한

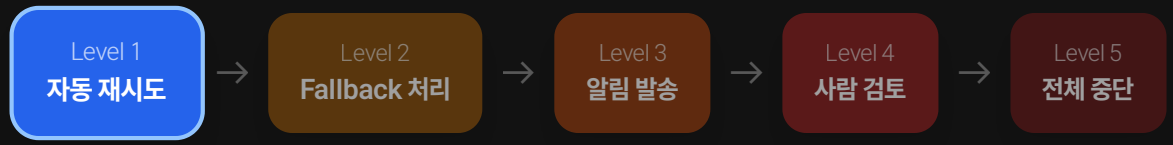
Agent는 실패할 수 있다 — **실패를 설계**해야 한다

에스컬레이션 전략

Level 1 **자동 재시도** → Level 2 **Fallback 처리** → Level 3 **알림 발송** → Level 4 **사람 검토** → Level 5 **전체 중단**

클릭하여 확인

에스컬레이션 전략



TimeoutError, ConnectionError → 자동 재시도 (지수 백오프)

에스컬레이션 전략

Level 1 **자동 재시도** → Level 2 **Fallback 처리** → Level 3 **알림 발송** → Level 4 **사람 검토** → Level 5 **전체 중단**

JSONDecodeError → Fallback 기본값 사용, **알 수 없는 에러** → 알림

에스컬레이션 전략

Level 1 **자동 재시도** → Level 2 **Fallback 처리** → Level 3 **알림 발송** → Level 4 **사람 검토** → Level 5 **전체 중단**

비즈니스 오류 → 사람 검토, **데이터 손상** → 전체 중단

기획서 작성 실수

기획서 작성 실수

① **Sub-task 경계가 모호함**

"데이터 처리" ✗ → "원시 JSON → 정규화 DataFrame 변환" ✓

기획서 작성 실수

① **Sub-task 경계가 모호함**

"데이터 처리" ✗ → "원시 JSON → 정규화 DataFrame 변환" ✓

② **성공 케이스만 정의**

"외부 API가 실패하면 어떻게 되는가?" 반드시 답하라

기획서 작성 실수

① **Sub-task 경계가 모호함**

"데이터 처리" ✗ → "원시 JSON → 정규화 DataFrame 변환" ✓

② **성공 케이스만 정의**

"외부 API가 실패하면 어떻게 되는가?" 반드시 답하라

③ **상태 저장 위치 미결정**

메모리? DB? Redis? → 재시작 후에도 유지돼야 하는가?

기획서 작성 실수

① **Sub-task 경계가 모호함**

"데이터 처리" ✗ → "원시 JSON → 정규화 DataFrame 변환" ✓

② **성공 케이스만 정의**

"외부 API가 실패하면 어떻게 되는가?" 반드시 답하라

③ **상태 저장 위치 미결정**

메모리? DB? Redis? → 재시작 후에도 유지돼야 하는가?

④ **입출력 타입 불일치**

Sub-task A 출력이 B 입력 타입과 맞아야 한다

❓ Q&A — Session 3

Q. Sub-task 분해는 얼마나 세분화해야 하는가?

❓ Q&A — Session 3

Q. Sub-task 분해는 얼마나 세분화해야 하는가?

> A. 독립적으로 테스트할 수 있는 최소 단위가 적당하다.
> "하나의 도구를 사용하는 단위"가 좋은 기준이다.

❓ Q&A — Session 3

Q. Sub-task 분해는 얼마나 세분화해야 하는가?

> A. 독립적으로 테스트할 수 있는 최소 단위가 적당하다.
> "하나의 도구를 사용하는 단위"가 좋은 기준이다.

❓ Q&A — Session 3

Q. Sub-task 분해는 얼마나 세분화해야 하는가?

> A. 독립적으로 테스트할 수 있는 최소 단위가 적당하다.
> "하나의 도구를 사용하는 단위"가 좋은 기준이다.

Q. 예외 처리를 미리 다 정의할 수 있는가?

> A. 모든 경우를 정의하기는 어렵다.
> "외부 의존성 실패", "데이터 오류", "LLM 응답 오류" 세 범주만 커버해도 80%를 처리할 수 있다.

🧪 Quiz — Sub-task 분해 원칙

다음 중 올바른 Sub-task 분해 원칙이 **아닌 것**은?

A) 하나의 Sub-task는 하나의 책임만 가진다

B) Sub-task는 독립적으로 테스트 가능해야 한다

C) 하나의 Sub-task에 가능한 많은 기능을 묶어 효율을 높인다

D) Sub-task 출력이 다음 Sub-task 입력이 된다

🧪 Quiz — Sub-task 분해 원칙

다음 중 올바른 Sub-task 분해 원칙이 **아닌 것**은?

A) 하나의 Sub-task는 하나의 책임만 가진다

B) Sub-task는 독립적으로 테스트 가능해야 한다

C) 하나의 Sub-task에 가능한 많은 기능을 묶어 효율을 높인다

D) Sub-task 출력이 다음 Sub-task 입력이 된다

정답: C

단일 책임 원칙(SRP) 위반 — 실패 시 어디서 문제가 생겼는지 파악 어렵다

🧪 Quiz — Stateful이 반드시 필요한 경우는?

A) 이메일 제목에서 카테고리를 추출 (건당 독립 처리)

B) PDF를 텍스트로 변환

C) 사용자와 대화하며 점진적으로 요구사항을 수집

D) 이미지에서 텍스트를 OCR

🧪 Quiz — Stateful이 반드시 필요한 경우는?

A) 이메일 제목에서 카테고리를 추출 (건당 독립 처리)

B) PDF를 텍스트로 변환

C) 사용자와 대화하며 점진적으로 요구사항을 수집

D) 이미지에서 텍스트를 OCR

정답: C

대화형 요구사항 수집은 이전 대화 내용을 기억해야 한다 — A, B, D는 Stateless로 충분

실습 — Lab 3: Agent 구조 다이어그램

- **강의** Agent 구조 패턴 + 다이어그램 요소 (30분)
- **시연** Excalidraw로 Planner형 다이어그램 구축 (25분)
- **과제** 개인 Agent 구조 다이어그램 완성 (45분)
- **공유** 발표 + 피드백 (20분)

labs/day1/s3-agent-blueprint/ — README.md 참고

☕ 쉬는 시간

10분 후 4교시를 시작합니다

4교시

MCP · RAG · Hybrid 구조 판단

Tool 설계 · 청킹 전략 · 구조 선택 기준

구조 선택이 시스템 한계를 결정한다

구조 선택이 시스템 한계를 결정한다

MCP, RAG, Hybrid — 세 구조는 각기 **다른 문제**를 해결한다

구조 선택이 시스템 한계를 결정한다

MCP, RAG, Hybrid — 세 구조는 각기 **다른 문제**를 해결한다

잘못된 구조 선택 → 기능 추가할수록 **복잡도만 증가**

구조 선택이 시스템 한계를 결정한다

MCP, RAG, Hybrid — 세 구조는 각기 **다른 문제**를 해결한다

잘못된 구조 선택 → 기능 추가할수록 **복잡도만 증가**

나중에 마이그레이션하는 비용은 처음부터 올바르게 설계하는 것보다 **5-10배**

구조 선택이 시스템 한계를 결정한다

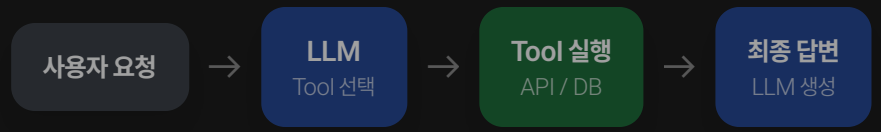
MCP, RAG, Hybrid — 세 구조는 각기 **다른 문제**를 해결한다

잘못된 구조 선택 → 기능 추가할수록 **복잡도만 증가**

나중에 마이그레이션하는 비용은 처음부터 올바르게 설계하는 것보다 **5-10배**

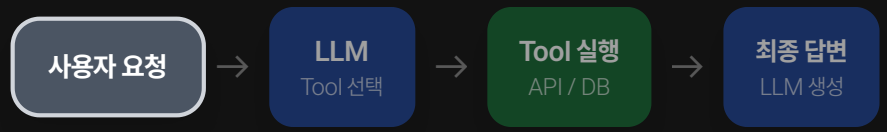
초기 설계 단계에서 올바른 구조를 선택해야 한다

MCP(Function Calling) 동작 원리



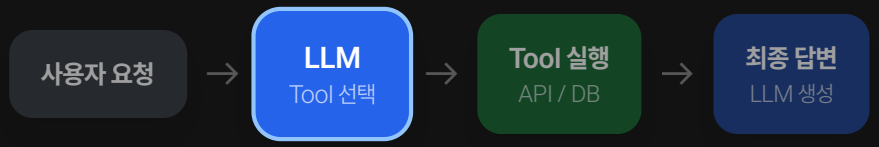
클릭하여 흐름 확인

MCP(Function Calling) 동작 원리



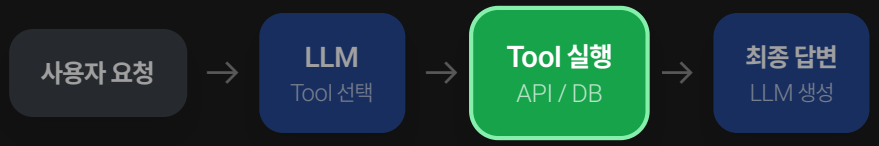
사용자가 자연어로 요청

MCP(Function Calling) 동작 원리



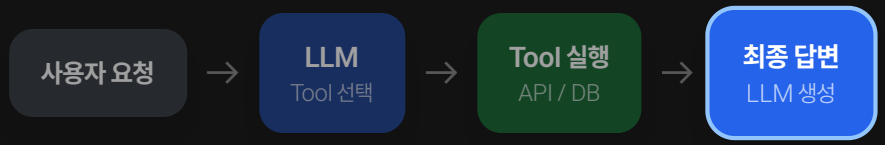
LLM이 **"어떤 Tool을 어떤 인자로 호출할지"** 명령 생성

MCP(Function Calling) 동작 원리



시스템이 실제 Tool 실행 → 결과를 LLM에 전달

MCP(Function Calling) 동작 원리



LLM이 Tool 결과를 기반으로**최종 자연어 답변**생성

좋은 Tool 설계 기준

좋은 Tool 설계 기준

① **단일 책임** — 하나의 Tool은 하나의 명확한 기능만 수행

✗ manage_data(action, data) → ✓ get_user_by_id(user_id)

좋은 Tool 설계 기준

① **단일 책임** — 하나의 Tool은 하나의 명확한 기능만 수행

✗ manage_data(action, data) → ✓ get_user_by_id(user_id)

② **명확한 Description** — LLM이 언제 이 Tool을 써야 하는지 이해

docstring이 Tool의 사용 가이드가 된다

좋은 Tool 설계 기준

① **단일 책임** — 하나의 Tool은 하나의 명확한 기능만 수행

✗ manage_data(action, data) → ✓ get_user_by_id(user_id)

② **명확한 Description** — LLM이 언제 이 Tool을 써야 하는지 이해

docstring이 Tool의 사용 가이드가 된다

③ **적절한 에러 응답** — Tool 실패 시 LLM이 이해할 수 있는 메시지

{"success": false, "error": "이유 설명"}

좋은 Tool 설계 기준

① **단일 책임** — 하나의 Tool은 하나의 명확한 기능만 수행

✗ manage_data(action, data) → ✓ get_user_by_id(user_id)

② **명확한 Description** — LLM이 언제 이 Tool을 써야 하는지 이해

docstring이 Tool의 사용 가이드가 된다

③ **적절한 에러 응답** — Tool 실패 시 LLM이 이해할 수 있는 메시지

{"success": false, "error": "이유 설명"}

Tool 수가 **15개 이상**이면 성능 문제 → 그룹화 또는 Sub-agent 분리

Tool 설계: 나쁜 예 vs 좋은 예

✗ 나쁜 예

```python
def manage_data(
    action: str,
    data: dict
) → dict:
    """데이터 관리:
    생성/조회/수정/삭제"""
    ...
```

문제: 책임이 4개, LLM이 언제 쓸지 모름

✓ 좋은 예

```python
def get_user_by_id(
    user_id: str
) → dict:
    """사용자 ID로 정보 조회"""
    ...

def update_user_email(
    user_id: str,
    new_email: str
) → dict:
    """사용자 이메일 변경"""
    ...
```

RAG 전체 흐름

색인 단계

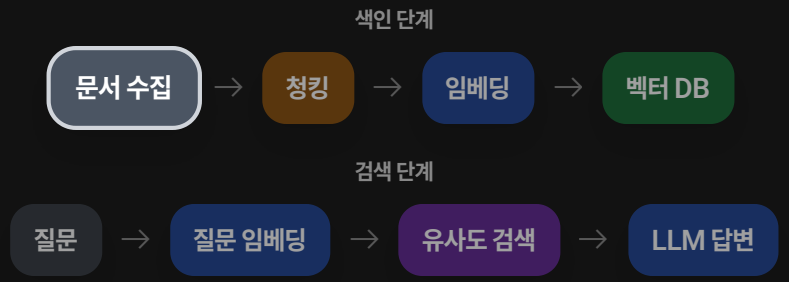
문서 수집 → 청킹 → 임베딩 → 벡터 DB

검색 단계

질문 → 질문 임베딩 → 유사도 검색 → LLM 답변

클릭하여 흐름 확인

RAG 전체 흐름



문서를 수집하여 RAG 파이프라인에 투입

RAG 전체 흐름

색인 단계

문서 수집 → 청킹 → 임베딩 → 벡터 DB

검색 단계

질문 → 질문 임베딩 → 유사도 검색 → LLM 답변

문서를 **청크로 분할** → **벡터로 변환** → DB에 저장

RAG 전체 흐름

색인 단계

문서 수집 → 청킹 → 임베딩 → 벡터 DB

검색 단계

질문 → 질문 임베딩 → 유사도 검색 → LLM 답변

사용자 질문이 들어오면 검색 시작

RAG 전체 흐름

색인 단계

문서 수집 → 청킹 → 임베딩 → 벡터 DB

검색 단계

질문 → 질문 임베딩 → 유사도 검색 → LLM 답변

질문을 임베딩 → **코사인 유사도**로 관련 청크 검색

RAG 전체 흐름

색인 단계

문서 수집 → 청킹 → 임베딩 → 벡터 DB

검색 단계

질문 → 질문 임베딩 → 유사도 검색 → LLM 답변

검색된 청크를 Context에 포함하여 LLM이 답변 생성

청킹 전략 비교

전략	방법	적합 문서	특징

청킹 전략 비교

전략	방법	적합 문서	특징
고정 크기	N 토큰마다 분할	균일한 텍스트	단순, 문장 끊길 수 있음

청킹 전략 비교

전략	방법	적합 문서	특징
고정 크기	N 토큰마다 분할	균일한 텍스트	단순, 문장 끊길 수 있음
문장 단위	문장 경계로 분할	일반 문서	의미 보존

청킹 전략 비교

전략	방법	적합 문서	특징
고정 크기	N 토큰마다 분할	균일한 텍스트	단순, 문장 끊길 수 있음
문장 단위	문장 경계로 분할	일반 문서	의미 보존
단락 단위	빈 줄 기준 분할	구조화된 문서	의미 단위 보존

청킹 전략 비교

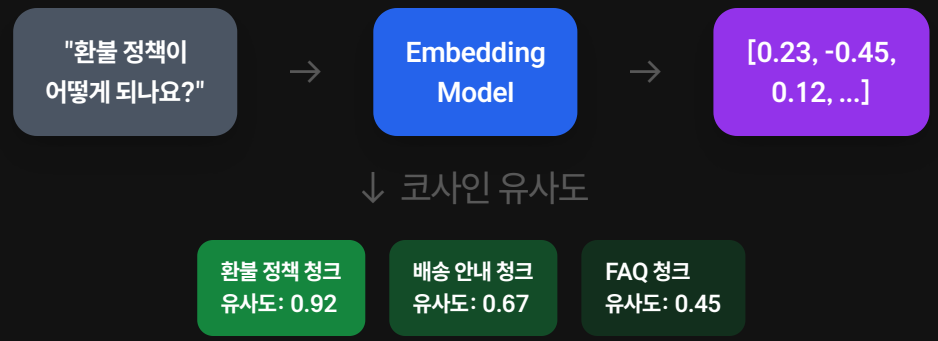
전략	방법	적합 문서	특징
고정 크기	N 토큰마다 분할	균일한 텍스트	단순, 문장 끊길 수 있음
문장 단위	문장 경계로 분할	일반 문서	의미 보존
단락 단위	빈 줄 기준 분할	구조화된 문서	의미 단위 보존
계층적	섹션 > 단락 > 문장	기술 문서	가장 정확

청킹 전략 비교

전략	방법	적합 문서	특징
고정 크기	N 토큰마다 분할	균일한 텍스트	단순, 문장 끊길 수 있음
문장 단위	문장 경계로 분할	일반 문서	의미 보존
단락 단위	빈 줄 기준 분할	구조화된 문서	의미 단위 보존
계층적	섹션 > 단락 > 문장	기술 문서	가장 정확

청킹 전략에 **정답은 없다** — 문서 유형에 따라 **실험**을 통해 최적값을 찾아라

임베딩과 유사도 검색


"환불 정책이 어떻게 되나요?"
Embedding Model
[0.23, -0.45, 0.12, ...]
↓ 코사인 유사도
환불 정책 청크 유사도: 0.92
배송 안내 청크 유사도: 0.67
FAQ 청크 유사도: 0.45

임베딩과 유사도 검색

"환불 정책이 어떻게 되나요?" → Embedding Model → [0.23, -0.45, 0.12, ...]

↓ 코사인 유사도

환불 정책 청크	배송 안내 청크	FAQ 청크
유사도: 0.92	유사도: 0.67	유사도: 0.45

Top-K 관련 청크를 검색하여 LLM Context에 주입

Tool 중심 vs RAG 중심

MCP (Tool 중심)

RAG 중심

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원

RAG 중심

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능

RAG 중심

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능

RAG 중심

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리

RAG 중심

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리
- Tool 수 많으면 선택 어려움

RAG 중심

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리
- Tool 수 많으면 선택 어려움

RAG 중심

- **지식 기반 Q&A** 강점

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리
- Tool 수 많으면 선택 어려움

RAG 중심

- **지식 기반 Q&A** 강점
- **근거 있는 답변**

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리
- Tool 수 많으면 선택 어려움

RAG 중심

- **지식 기반 Q&A** 강점
- **근거 있는 답변**
- 정적 문서에 적합

Tool 중심 vs RAG 중심

MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리
- Tool 수 많으면 선택 어려움

RAG 중심

- **지식 기반 Q&A** 강점
- **근거 있는 답변**
- 정적 문서에 적합
- **실시간 데이터** 미지원

Tool 중심 vs RAG 중심

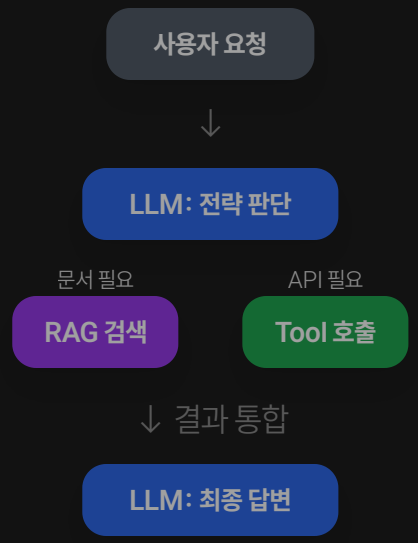
MCP (Tool 중심)

- **실시간 데이터** 지원
- **외부 시스템 조작** 가능
- 계산·변환 작업 가능
- **비정형 Q&A**에 불리
- Tool 수 많으면 선택 어려움

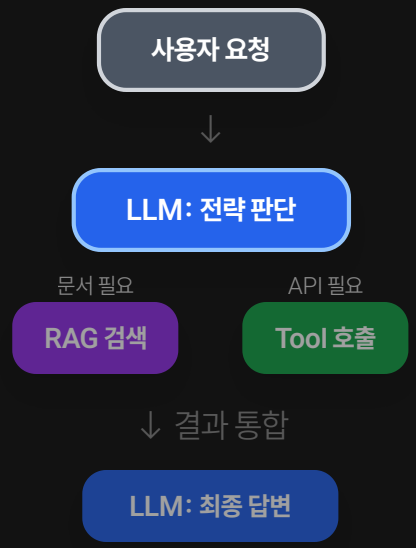
RAG 중심

- **지식 기반 Q&A** 강점
- **근거 있는 답변**
- 정적 문서에 적합
- **실시간 데이터** 미지원
- 검색 품질에 전적으로 의존

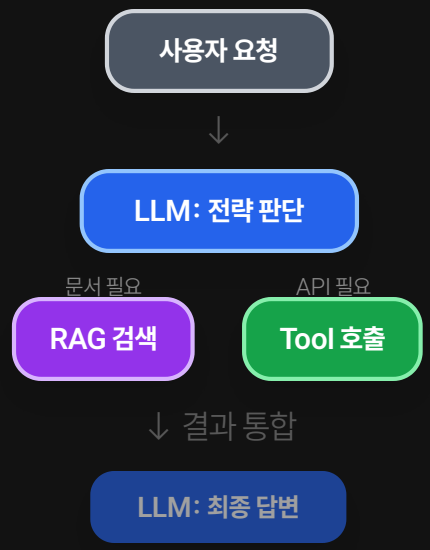
Hybrid 구조



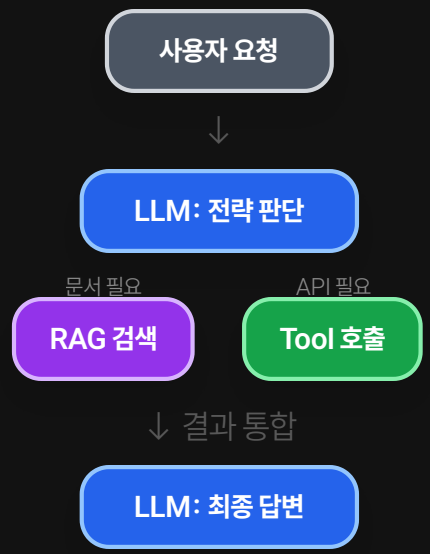
Hybrid 구조



Hybrid 구조



Hybrid 구조



RAG와 Tool을 **함께 사용** — Agent가 Tool의 일부로 RAG를 사용

Hybrid 설계 결정 기준

기준	RAG 비중 높임	Tool 비중 높임
데이터 변경 빈도	낮음 (문서)	**높음 (실시간)**
정확도 요구	근거 문서 필요	최신 데이터 필요
비용	임베딩 비용	API 호출 비용
응답 속도	상대적 빠름	API 지연 있음

Hybrid 설계 결정 기준

기준	RAG 비중 높임	Tool 비중 높임
데이터 변경 빈도	낮음 (문서)	**높음 (실시간)**
정확도 요구	근거 문서 필요	최신 데이터 필요
비용	임베딩 비용	API 호출 비용
응답 속도	상대적 빠름	API 지연 있음

Hybrid는 **두 구조의 복잡도 합산** — 충분한 이유가 있을 때만 선택

세 구조 종합 비교

항목	MCP (Tool)	RAG	Hybrid
실시간 데이터	✓	✗	✓
지식 Q&A	제한적	**강함**	**강함**
외부 조작	**강함**	✗	**강함**
구현 복잡도	중간	중간	**높음**
운영 비용	API 비용	임베딩 비용	**높음**
정확도	Tool 의존	검색 의존	높은 편

구조 선택 실수

구조 선택 실수

① **Hybrid를 기본 선택으로 삼는 것**

RAG나 Tool 단독으로 해결되는지 먼저 확인

구조 선택 실수

① **Hybrid를 기본 선택으로 삼는 것**

RAG나 Tool 단독으로 해결되는지 먼저 확인

② **Tool 수를 과도하게 늘리는 것**

15개 이상 시 성능 저하 → 그룹화 또는 Sub-agent

구조 선택 실수

① **Hybrid를 기본 선택으로 삼는 것**

RAG나 Tool 단독으로 해결되는지 먼저 확인

② **Tool 수를 과도하게 늘리는 것**

15개 이상 시 성능 저하 → 그룹화 또는 Sub-agent

③ **청킹 크기를 기본값으로 방치**

문서 유형에 따라 최적 크기가 다르다

구조 선택 실수

① **Hybrid를 기본 선택으로 삼는 것**

RAG나 Tool 단독으로 해결되는지 먼저 확인

② **Tool 수를 과도하게 늘리는 것**

15개 이상 시 성능 저하 → 그룹화 또는 Sub-agent

③ **청킹 크기를 기본값으로 방치**

문서 유형에 따라 최적 크기가 다르다

④ **임베딩 모델 미스매치**

색인·검색 시 반드시 같은 모델 사용

❓ Q&A — Session 4

Q. Tool 수가 많아지면 어떻게?

❓ Q&A — Session 4

Q. Tool 수가 많아지면 어떻게?

> A. 라우팅 레이어 추가: 첫 번째 LLM이 Tool 그룹을 판단하고, 두 번째 LLM이 구체적 Tool을 선택.
> 또는 도메인별로 Sub-agent를 분리하여 각각이 관련 Tool만 관리.

❓ Q&A — Session 4

Q. Tool 수가 많아지면 어떻게?

> A. 라우팅 레이어 추가: 첫 번째 LLM이 Tool 그룹을 판단하고, 두 번째 LLM이 구체적 Tool을 선택.
> 또는 도메인별로 Sub-agent를 분리하여 각각이 관련 Tool만 관리.

❓ Q&A — Session 4

Q. Tool 수가 많아지면 어떻게?

> A. 라우팅 레이어 추가: 첫 번째 LLM이 Tool 그룹을 판단하고, 두 번째 LLM이 구체적 Tool을 선택.
> 또는 도메인별로 Sub-agent를 분리하여 각각이 관련 Tool만 관리.

Q. 벡터 DB는 어떤 것을 써야?

> A. 소규모(100만 이하): pgvector로 시작. 대규모: Pinecone, Weaviate.
> 가장 중요한 것은 임베딩 모델 선택 — 벡터 DB는 나중에 교체 가능.

🧪 Quiz — 구조 선택

"고객 서비스 Agent: 제품 매뉴얼(PDF 500p)에서 답변을 찾고, 필요 시 주문 시스템에서 고객 주문 현황을 조회한다."

A) MCP만 사용

B) RAG만 사용

C) Hybrid (RAG + MCP)

D) 단순 LLM 호출

🧪 Quiz — 구조 선택

"고객 서비스 Agent: 제품 매뉴얼(PDF 500p)에서 답변을 찾고, 필요 시 주문 시스템에서 고객 주문 현황을 조회한다."

A) MCP만 사용

B) RAG만 사용

C) Hybrid (RAG + MCP)

D) 단순 LLM 호출

정답: C (Hybrid)

매뉴얼 검색 → RAG (정적 문서) · 주문 현황 조회 → MCP Tool (실시간 DB)

🧪 Quiz — RAG가 잘 처리하지 못하는 경우

A) "환불 정책이 어떻게 되나요?" (정책 문서 기반)

B) "제 주문 #12345의 현재 배송 위치는?" (실시간 추적)

C) "제품 설치 방법을 알려주세요" (매뉴얼 기반)

D) "이전 고객 문의에서 비슷한 케이스를 찾아주세요"

🧪 Quiz — RAG가 잘 처리하지 못하는 경우

A) "환불 정책이 어떻게 되나요?" (정책 문서 기반)

B) "제 주문 #12345의 현재 배송 위치는?" (실시간 추적)

C) "제품 설치 방법을 알려주세요" (매뉴얼 기반)

D) "이전 고객 문의에서 비슷한 케이스를 찾아주세요"

정답: B

실시간 배송 위치는 배송 API를 통해 조회해야 한다 — RAG는 이미 임베딩된 문서만 검색 가능

실습 — Lab 4: MCP vs RAG 비교

- **강의** MCP와 RAG 개념 · 동작 원리 (25분)
- **시연** PydanticAI로 MCP/RAG 각각 구현 시연 (25분)
- **과제** MCP/RAG 비교 구현 + 의사결정 매트릭스 (40분)
- **같이 보기** 온보딩 도우미 Agent 시나리오 토론 (15분)

labs/day1/s4-architecture-decision/ — README.md 참고

시연: MCP 방식 — Tool 호출 Agent

```python
# Agent 생성
agent = Agent(
    "openai:gpt-5.4",
    instructions="TechShop 고객 상담 에이전트. Tool로 정확한 정보 제공.",
)

# Tool 등록 — docstring이 LLM의 Tool 선택 기준
@agent.tool_plain
def get_refund_policy(product_id: str) → str:
    """제품별 환불 정책을 조회한다. 환불 기간, 조건, 방법을 반환."""
    return DATA["refund_policies"][product_id]

@agent.tool_plain
def get_order_status(order_id: str) → str:
    """주문 번호로 배송 현황, 예상 도착일을 조회."""
    return DATA["orders"][order_id]
```

시연: MCP 방식 — Tool 호출 Agent

```
agent = Agent(
    "openai:gpt-5.4",
    instructions="TechShop 고객 상담 에이전트. Tool로 정확한 정보 제공.",
)

# Tool 등록 — docstring이 LLM의 Tool 선택 기준
@agent.tool_plain
def get_refund_policy(product_id: str) → str:
    """제품별 환불 정책을 조회한다. 환불 기간, 조건, 방법을 반환."""
    return DATA["refund_policies"][product_id]

@agent.tool_plain
def get_order_status(order_id: str) → str:
    """주문 번호로 배송 현황, 예상 도착일을 조회."""
    return DATA["orders"][order_id]

# 실행 — LLM이 질문 분석 → 적절한 Tool 자동 선택·호출
result = await agent.run("PROD-001 환불 정책이 어떻게 되나요?")
```

시연: MCP 방식 — Tool 호출 Agent

```
agent = Agent(
    "openai:gpt-5.4",
    instructions="TechShop 고객 상담 에이전트. Tool로 정확한 정보 제공.",
)

# Tool 등록 — docstring이 LLM의 Tool 선택 기준
@agent.tool_plain
def get_refund_policy(product_id: str) → str:
    """제품별 환불 정책을 조회한다. 환불 기간, 조건, 방법을 반환."""
    return DATA["refund_policies"][product_id]

@agent.tool_plain
def get_order_status(order_id: str) → str:
    """주문 번호로 배송 현황, 예상 도착일을 조회."""
    return DATA["orders"][order_id]

# 실행 — LLM이 질문 분석 → 적절한 Tool 자동 선택·호출
result = await agent.run("PROD-001 환불 정책이 어떻게 되나요?")
```

시연: MCP 방식 — Tool 호출 Agent

```
agent = Agent(
    "openai:gpt-5.4",
    instructions="TechShop 고객 상담 에이전트. Tool로 정확한 정보 제공.",
)

# Tool 등록 — docstring이 LLM의 Tool 선택 기준
@agent.tool_plain
def get_refund_policy(product_id: str) → str:
    """제품별 환불 정책을 조회한다. 환불 기간, 조건, 방법을 반환."""
    return DATA["refund_policies"][product_id]

@agent.tool_plain
def get_order_status(order_id: str) → str:
    """주문 번호로 배송 현황, 예상 도착일을 조회."""
    return DATA["orders"][order_id]

# 실행 — LLM이 질문 분석 → 적절한 Tool 자동 선택·호출
result = await agent.run("PROD-001 환불 정책이 어떻게 되나요?")
```

복합 질문 → LLM이 **여러 Tool을 순서대로 자동 호출**

시연: RAG 방식 — 문서 검색 Agent

```python
# 1. 문서 청킹 + 임베딩
chunks = split_document(doc_text, chunk_size=300)
embeddings = [embed(chunk) for chunk in chunks]

# 2. 유사도 검색 함수
def search(query: str, top_k: int = 3) -> list[str]:
    q_emb = embed(query)
    sims = [cosine_similarity(q_emb, e) for e in embeddings]
    ranked = sorted(zip(chunks, sims), key=lambda x: x[1], reverse=True)
    return [c for c, _ in ranked[:top_k]]

# 3. 검색 결과를 Agent instructions에 주입
relevant = search("환불 정책이 어떻게 되나요?")
context = "\n\n---\n\n".join(relevant)

rag_agent = Agent("openai:gpt-5.4",
    instructions=f"다음 문서를 바탕으로 답변.\n문서에 없는 내용은 모른다고 답하라.\n\n{context}"
)
result = await rag_agent.run("환불 정책이 어떻게 되나요?")
```

시연: RAG 방식 — 문서 검색 Agent

```python
# 1. 문서 청킹 + 임베딩
chunks = split_document(doc_text, chunk_size=300)
embeddings = [embed(chunk) for chunk in chunks]

# 2. 유사도 검색 함수
def search(query: str, top_k: int = 3) -> list[str]:
    q_emb = embed(query)
    sims = [cosine_similarity(q_emb, e) for e in embeddings]
    ranked = sorted(zip(chunks, sims), key=lambda x: x[1], reverse=True)
    return [c for c, _ in ranked[:top_k]]

# 3. 검색 결과를 Agent instructions에 주입
relevant = search("환불 정책이 어떻게 되나요?")
context = "\n\n---\n\n".join(relevant)

rag_agent = Agent("openai:gpt-5.4",
    instructions=f"다음 문서를 바탕으로 답변.\n문서에 없는 내용은 모른다고 답하라.\n\n{context}"
)
result = await rag_agent.run("환불 정책이 어떻게 되나요?")
```

시연: RAG 방식 — 문서 검색 Agent

```python
# 1. 문서 청킹 + 임베딩
chunks = split_document(doc_text, chunk_size=300)
embeddings = [embed(chunk) for chunk in chunks]

# 2. 유사도 검색 함수
def search(query: str, top_k: int = 3) -> list[str]:
    q_emb = embed(query)
    sims = [cosine_similarity(q_emb, e) for e in embeddings]
    ranked = sorted(zip(chunks, sims), key=lambda x: x[1], reverse=True)
    return [c for c, _ in ranked[:top_k]]

# 3. 검색 결과를 Agent instructions에 주입
relevant = search("환불 정책이 어떻게 되나요?")
context = "\n\n---\n\n".join(relevant)

rag_agent = Agent("openai:gpt-5.4",
    instructions=f"다음 문서를 바탕으로 답변.\n문서에 없는 내용은 모른다고 답하라.\n\n{context}"
)
result = await rag_agent.run("환불 정책이 어떻게 되나요?")
```

MCP vs RAG — 같은 질문, 다른 방식

MCP (Tool 호출)

질문 → LLM 판단

[Tool] get_refund_policy("PROD-001")

→ 정확한 정책 데이터 반환

[LLM] 자연어 답변 생성

장점: 실시간, 정확

단점: 사전 Tool 설계 필요

RAG (문서 검색)

질문 → 임베딩

[검색] 유사도 0.92 청크 3개

→ Context에 청크 주입

[LLM] 문서 기반 답변 생성

장점: 근거 기반, 확장 쉬움

단점: 실시간 데이터 불가

Day 1 핵심 키워드

Day 1 핵심 키워드

Pain → Tool

문제에서 출발하라
기술에서 출발하지 마라

Day 1 핵심 키워드

Pain → Tool

문제에서 출발하라
기술에서 출발하지 마라

프롬프트 전략

Zero-shot → Few-shot → CoT
비용과 정확도의 트레이드오프

Day 1 핵심 키워드

Pain → Tool

문제에서 출발하라
기술에서 출발하지 마라

프롬프트 전략

Zero-shot → Few-shot → CoT
비용과 정확도의 트레이드오프

구조화된 기획

Sub-task 분해 + IPO 명세
실패를 설계하라

Day 1 핵심 키워드

Pain → Tool

문제에서 출발하라
기술에서 출발하지 마라

프롬프트 전략

Zero-shot → Few-shot → CoT
비용과 정확도의 트레이드오프

구조화된 기획

Sub-task 분해 + IPO 명세
실패를 설계하라

MCP · RAG · Hybrid

단순한 구조부터 시작
Hybrid는 충분한 이유 필요

Day 1 산출물 점검

Lab	산출물	확인
1	Agent 후보 도출 워크시트 2-3매	□
2	컨텍스트 전략 비교 코드 (정확도 비교)	□
3	Agent 구조 다이어그램 1매	□
4	MCP vs RAG 비교 코드 + 구조 설계 문서	□

Day 1 산출물 점검

Lab	산출물	확인
1	Agent 후보 도출 워크시트 2-3매	☐
2	컨텍스트 전략 비교 코드 (정확도 비교)	☐
3	Agent 구조 다이어그램 1매	☐
4	MCP vs RAG 비교 코드 + 구조 설계 문서	☐

이 산출물들은 **Day 2부터의 코드 구현**에서 직접 참조됩니다

Day 2 예고

Agent 제어 흐름 설계 & 상태 관리

LangGraph · Tool Validation · 구조 리팩토링